

- 位运算
- 二进制集合操作
- 快速幂&快速乘
- 置换和排列
- 数论
 - 素数
 - 最大公因数
 - 数论分块
 - 欧拉函数&欧拉定理
 - 筛法
 - 分解质因数&约数倍数
 - 裴蜀定理
 - 线性同余方程&乘法逆元
 - 中国剩余定理
 - 二次剩余
 - 离散对数
 - 莫比乌斯反演
 - 威尔逊定理
 - Min_25筛
- 多项式与生成函数
 - 快速傅里叶变换
 - 多项式牛顿迭代
 - 多项式多点求值|快速插值

- 组合数学
 - 排列组合
 - 鸽巢原理
 - 容斥原理
 - 康托展开
 - 斐波那契数列
 - 错位排列
 - 卡特兰数
 - 斯特林数
- 线性代数
 - 线性基
 - 矩阵
 - 矩阵快速幂
 - 高斯消元
- 概率论
 - 概率与期望
 - 条件概率与独立性
- 博弈论
 - 公平组合游戏
 - 非公平组合游戏
 - 反常游戏
 - 序理论

位运算

$a \oplus b \geq a - b, (a \geq b)$

$a \& b + a \mid b = a + b$

$a \mid b - a \& b = a \oplus b$

获取a的第b位: `a >> b & 1`

将a的第b位置0: `a & ~(int)1 << b`

将a的第b位置1: `a | ((int)1 << b)`

将a的第b位取反: `a ^ ((int)1 << b)`

按汉明权重递增枚举: 汉明权重 $\Leftrightarrow$ `popcount` , 时间复杂度 $O(n)$

```

1  for (int i = 0; (1ll << i) - 1 <= n; i++) { // 特判了 0, 因为 0 无后继
2      for (int x = (1ll << i) - 1, t; x <= n; t = x + (x & -x), x = x ? (t | (((t & -t) / (x & -x)) >> 1) - 1)) :
        (n + 1)) {
3      }
4  }

```

结论: $\bigoplus_{i=1}^n i = \begin{cases} 1 & n \bmod 4 = 1 \\ n+1 & n \bmod 4 = 2 \\ 0 & n \bmod 4 = 3 \\ n & n \bmod 4 = 0 \end{cases}$

内置函数:

1. `int __builtin_ffs(int x)` : 返回 x 的二进制末尾最后一个1的位置, 位置的编号从1开始(最低位编号为1)。当 x 为0时返回0
2. `int __builtin_clz(unsigned int x)` : 返回 x 的二进制的前导0的个数, 当 x 为0时, 结果未定义
3. `int __builtin_ctz(unsigned int x)` : 返回 x 的二进制末尾连续0的个数, 当 x 为0时, 结果未定义
4. `int __builtin_clrsb(int x)` : 当 x 的符号位为0时返回 x 的二进制的前导0的个数减一, 否则返回 x 的二进制的前导1的个数减一
5. `int __builtin_popcount(unsigned int x)` : 返回 x 的二进制中1的个数
6. `int __builtin_parity(unsigned int x)` : 判断 x 的二进制中1的个数的奇偶性

这些函数都可以在函数名末尾添加 `l` 或 `ll` (如 `__builtin_popcountll`) 来使参数类型变为 `(unsigned) long` 或 `(unsigned) long long`

不考虑0求出一个数 n 以2为底的对数, `int` 使用 `31 - __builtin_clz(n)`, `long long` 使用 `63 - __builtin_clzll(n)`

- 求出大于 $x \geq 1$ 的最小的2的幂次方数, 使用 `1ll << (64 - __builtin_clzll(x))`
- 求出大于等于 $x \geq 2$ 的最小的2的幂次方数, 使用 `1ll << (64 - __builtin_clzll(x - 1))`

二进制集合操作

操作:

- 交集: $a \cap b \Leftrightarrow a \& b$
- 并集: $a \cup b \Leftrightarrow a | b$
- 补集: $\bar{a} \Leftrightarrow \sim a$ (全集为二进制全是1)
- 差集: $a \setminus b \Leftrightarrow a \& (\sim b)$
- 对称差: $a \Delta b \Leftrightarrow a \oplus b$

结论:

1. $a \& b = 0 \Leftrightarrow a \cap b = \emptyset \Leftrightarrow a \cap \bar{b} = a \Leftrightarrow a \setminus b = a \Leftrightarrow a \subseteq \bar{b}$
2. $a \& b \neq 0 \Leftrightarrow a \cap b \neq \emptyset \Leftrightarrow a \cap \bar{b} \subsetneq a \Leftrightarrow a \setminus b \subsetneq a \Leftrightarrow a \not\subseteq \bar{b}$ (1.与2.互相容斥)

技巧:

- 模2的次幂:

```

1  int bitmod = 1ll << bit - 1;
2  x &= bitmod;

```

- 判断是否为2的非负整数次幂: `x > 0 && (x & (x - 1)) == 0`
- 遍历一个集合的子集: 使用位运算降序遍历子集

```

1  for (int state = mask; state; state = (state - 1) & mask) // 不会遍历空集
2  for (int state = mask; ~state; state = state ? (state - 1) & mask : (int)-1) // 会遍历空集

```

- 遍历所有集合的子集: 时间复杂使用二项式定理证明 $O(3^n)$

```

1  for (int mask = 0; mask < (1ll << n); mask++) {
2      for (int state = mask; state; state = (state - 1) & mask) // 不会遍历空集
3      }
4  for (int mask = 0; mask < (1ll << n); mask++) {
5      for (int state = mask; ~state; state = state ? (state - 1) & mask : (int)-1) // 会遍历空集
6      }

```

快速幂&快速乘

快速幂：以二进制思维计算乘幂,时间复杂度 $O(\log n)$ ，模数质数见 [乘法逆元](#)

```
1  template <ll mod = mod> ll binpow(ll a, ll b, ll res = 1) {
2      for (a %= mod; b >= 1, (a *= a) %= mod) {
3          if (b & 1) (res *= a) %= mod;
4      }
5      return res;
6  }
```

应用：

多次置换问题：对长度为 n 的序列，使用同一置换置换 k 次

- 1. 对每个置换位置做 k 次幂，时间复杂度 $O(n \log k)$
- 2. 对置换建图，找环，在每个环找 k 次幂（让 k 模上环长）可以达到 $O(n)$

加速几何中对点集的操作：对于三维点集，1. 按向量移动点位置。2. 按比例缩放点坐标。3. 绕坐标轴旋转。 m 次变换，每次变换可重复操作 k 次

- 构建为向量 $[x, y, z, 1]^T$ ，每个变换可以被表示为 4×4 的矩阵，向量多了一维是对 (1) 操作的描述，矩阵快速幂即可对 n 个点做相同的所有变换，时间复杂度 $O(n + m \log k)$

定长路径计数：有向图边权为 1 中，求任意俩点长度为 k 的路径数，将邻接表取 k 次幂后得到 $Ans_{u,v}$ 即为 u 到 v 长度为 k 的路径数目，复杂度 $O(n^3 \log k)$

模意义下大整数乘法：

龟速乘：以二进制思维计算数乘,用于数乘过大时(或直接用 `__int128`),时间复杂度 $O(\log n)$

快速乘： $a \times b \bmod mod = a \times b - \lfloor \frac{ab}{mod} \rfloor \times mod = (a \times b - \lfloor \frac{ab}{mod} \rfloor \times mod) \bmod 2^{64}$

- **快速乘代码** 见 [Pollard-Rho算法](#)

置换和排列

对换：交换 i 和 j 的位置

定理：任意一个排列可由若干个对换构成，对于，每一次对换都会改变排列的奇偶性

数论

素数

素数性质：所有大于 3 素数可以表示为 $6n \pm 1$

素数计数函数：小于等于 x 的素数个数为 $\pi(x) \sim \frac{x}{\ln(x)}$

Carmichael 函数： $\forall n \in \mathbb{N}, \lambda n = \max\{\delta_n(a) : (a, n) = 1\} \Leftrightarrow \forall a \in \mathbb{N}, \gcd(a, n) = 1, s.t. a^m \equiv 1 \pmod n$ 的最小 m

- **Carmichael 函数性质：**

1. **Carmichael 定理：** $\forall p \in primes, r \in \mathbb{N} \Rightarrow \lambda(p^r) = \begin{cases} \frac{1}{2}\varphi(p^r), & p = 2 \wedge r \geq 3 \\ \varphi(p^r), & otherwise \end{cases} \Rightarrow$
 1. $\forall n \in \mathbb{N} \Rightarrow \lambda(n) \mid \varphi(n)$
 2. $\forall a, b \in \mathbb{N}, a \mid b \Rightarrow \lambda(a) \mid \lambda(b)$
2. **结合唯一分解定理：** $\forall n = \prod_{i=1}^k p_i^{r_i} \in \mathbb{N} \Rightarrow \lambda(n) = [\lambda(p_1^{r_1}), \lambda(p_2^{r_2}), \dots, \lambda(p_k^{r_k})] = \text{lcm}(\lambda(p_1^{r_1}), \lambda(p_2^{r_2}), \dots, \lambda(p_k^{r_k})) \Rightarrow$
 1. $\forall a, b \in \mathbb{N} \Rightarrow \lambda([a, b]) = [\lambda(a), \lambda(b)] = \text{lcm}(\lambda(a), \lambda(b))$

Carmichael 数： $\forall n \notin primes, (\forall a \in \mathbb{N}, \gcd(a, n) = 1 \Rightarrow a^{n-1} \equiv 1 \pmod n) \Leftrightarrow n$ 为 *Carmichael* 数，最小的为 $561 = 3 \times 11 \times 17$

- **判别 Carmichael 数：**

1. **Korselt 判别法：**（合数 n 无平方因子 $\wedge \forall p \in primes, p \mid n \Rightarrow p-1 \mid n-1$ ） $\Leftrightarrow n$ 为 *Carmichael* 数
2. **化简 Korselt：**合数 $n, \lambda(n) \mid n-1 \Leftrightarrow n$ 为 *Carmichael* 数

- **Carmichael 数性质：**

1. *Carmichael* 数无平方因子且至少有 3 个不同的质因数
2. 设 $C(n)$ 为小于等于 n 的 *Carmichael* 数个数, $n^{2/7} < C(n) < n \exp(-c \frac{\ln n \ln \ln \ln n}{\ln \ln n})$ ，其中 c 为常数, $C(10^9) = 646, C(10^{18}) = 1401644$

Miller-Rabin 素性测试: 时间复杂度为 $O(k \log n)$, k 为数 n 测试的轮次

- **结论:** 对数 n 最多只需要测试 $[2, \min\{n-2, \lfloor 2 \ln^2 n \rfloor\}]$ 全部整数即可确定 n 的素性
- **基底测试:** 所以基底都得测试一遍, 先将 a 替换为 $a \bmod n$, 若 $a \equiv 0 \pmod n$ 或 $a \equiv \pm 1 \pmod n$ 则不用测试
 1. **对于 $[1, 2^{32}]$ 的数:** 只需测试基底为 $\{2, 7, 61\}$ 即可确定 n 的素性
 2. **对于 $[1, 2^{64}]$ 的数:** 只需测试基底为 $\{2, 325, 9375, 28178, 450775, 9780504, 1795265022\}$ 或 $\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37\}$ (前 12 个素数) 即可确定 n 的素性
 3. **基底测试版代码:** 见 [Pollard-Rho算法](#), 此处只有随机数版

```
1 auto MillerRabin_rd = [](ll n, ll test_time = 10) -> bool { // test_time 为测试次数, 建议不小于 8, 也不宜过大
2     if (n < 4 || n % 2 == 0 || n % 3 == 0) return n == 2 || n == 3;
3     for (ll t = __builtin_ctzll(n - 1), u = (n - 1) >> t; test_time; test_time--) {
4         ll x = rd() % (n - 3) + 2, v = binpow(x, u, n), s = 0; // 随机的取值范围应为 [2, n - 2]
5         if (v == 1) continue; // 通过此次测试
6         for (; s < t; s++, v = binmul(v, v, n)) {
7             if (v == n - 1) break; // 得到平凡平方根 n - 1
8         }
9         if (s == t) return false; // 找到非平凡平方根或无法通过 Fermat 素数测试
10    }
11    return true;
12};
```

反素数: $\forall n \in \mathbb{N}, (\forall x < n, x \in \mathbb{N} \Rightarrow d(x) < d(n)) \Leftrightarrow n$ 为反素数

- **性质:** $\forall n = \prod_{i=1}^k p_i^{r_i} \in \mathbb{N}$ 为反素数 $\Rightarrow$
 1. n 是从 2 开始的连续素数的幂次乘积
 2. $r_1 \geq r_2 \geq r_3 \geq \dots \geq r_k$
- 3. 使用暴力 dfs 求解
- **题型:** 1. 求因子数一定的最小数。2. 求 n 以内因子数最多的数。

最大公因数

永远的神, 有事没事就找他, 很奇妙的加减就想他, 原数组不能用就用差分数组

最大公约数性质: $\gcd(a, b) = \gcd(a - b, b) \leq a - b, (a \geq b)$

- $a_1, \dots, a_n \in \{\mathbb{Z}\}$ 俩俩互质且 $\exists v \in \mathbb{Z}, s.t. \prod_{i=1}^n a_i = v^m \Rightarrow \forall i \in [1, n], \sqrt[n]{a_i} \in \mathbb{Z}$

最小公倍数性质: $[a, b, c][ab, bc, ca] = [a, b][b, c][c, a]$

1. $(ab, bc, ca)[a, b, c] = |abc|$
2. $\frac{(a, b, c)^2}{(a, b)(b, c)(a, c)} = \frac{[a, b, c]^2}{[a, b][b, c][a, c]}$

时间复杂度 $O(\log n)$

```
1 int gcd(int a, int b) {
2     return b == 0 ? a : gcd(b, a % b);
3 }
4 int lcm(int a, int b) {
5     return a / gcd(a, b) * b;
6 }
```

Stein算法求gcd: 稍微快的 $O(\log)$

1. $x = y \Rightarrow \gcd(x, y) = x = y$
2. $x = 0 \Rightarrow \gcd(x, y) = y$
3. $x \equiv y \equiv 0 \pmod 2 \Rightarrow \gcd(x, y) = 2 \times \gcd(\frac{x}{2}, \frac{y}{2})$
4. $x \equiv 0 \pmod 2 \wedge y \equiv 1 \pmod 2 \Rightarrow \gcd(x, y) = \gcd(\frac{x}{2}, y)$
5. $x \equiv y \equiv 1 \pmod 2 \Rightarrow \gcd(x, y) = \gcd(x - y, y)$

```
1 auto gcd = [](int a, int b) {
2     int az = __builtin_ctzll(a), bz = __builtin_ctzll(b), z = min(az, bz), dif;
3     for (b >>= bz; a; b = min(a, b), a = abs(dif)) {
4         a >>= az, dif = b - a, az = __builtin_ctzll(dif);
5     }
6     return b << z;
7 };
```

扩展欧几里得算法： 求 $ax + by = \gcd(a, b)$ 的一组解，时间复杂度 $O(\log n)$

- $ax_1 + by_1 = \gcd(a, b) = \gcd(b, a \bmod b) = bx_2 + (a \bmod b)y_2 = bx_2 + (a - \lfloor \frac{a}{b} \rfloor b)y_2 = ay_2 + b(x_2 - \lfloor \frac{a}{b} \rfloor y_2)$
- $\Rightarrow \begin{cases} x_1 = y_2 \\ y_1 = x_2 - \lfloor \frac{a}{b} \rfloor y_2 \end{cases}$
- $exgcd$ 求出的解 $|x| \leq b, |y| \leq a$
- 矩阵的解释：** $\gcd(a, b) = \gcd(b, a \bmod b) \Leftrightarrow \begin{bmatrix} b & \\ a \bmod b & \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & -\lfloor \frac{a}{b} \rfloor \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} \Rightarrow \begin{bmatrix} \gcd(a, b) \\ 0 \end{bmatrix} = (\cdots \begin{bmatrix} 0 & 1 \\ 1 & -\lfloor \frac{a}{b} \rfloor \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}) \begin{bmatrix} a \\ b \end{bmatrix}$
 - 矩阵版代码：** 见 [线性同余方程](#)，此处只有递归版

```
1 auto exgcd = [](auto && self, int a, int b, int & x, int & y) -> int {
2     if (!b) { // 递归版
3         x = 1, y = 0;
4         return a;
5     }
6     int d = self(self, b, a % b, y, x);
7     y -= (a / b) * x; //默认向下取整
8     return d;
9 };
```

数论分块

引理： $\forall a, b, c \in \mathbb{Z} \Rightarrow \lfloor \frac{a}{bc} \rfloor = \lfloor \frac{\lfloor \frac{a}{b} \rfloor}{c} \rfloor$ ，且 $\forall n \in \mathbb{N}_+ \Rightarrow |\{ \lfloor \frac{n}{d} \rfloor \mid d \in \mathbb{N}_+, d \leq n \}| \leq 2\sqrt{n}$

数论分块： 快速计算形如 $\sum_{i=1}^n f(i)g(\lfloor \frac{n}{i} \rfloor)$ 的和式，当能 $O(1)$ 算 $f(r) - f(l)$ 或预处理 f 的前缀和时，数论分块的时间复杂度为 $O(\sqrt{n})$

- 数论分块结论：** 下取整 $\lfloor \frac{n}{i} \rfloor$ 所在块的右端点为 $\lfloor \frac{n}{\lfloor \frac{n}{i} \rfloor} \rfloor$ ，上取整 $\lceil \frac{n}{i} \rceil$ 所在块的右端点为 $\lfloor \frac{n-1}{\lceil \frac{n}{i} \rceil} \rfloor$
 - 注意：** 实际使用时，枚举区间不一定与块的右端点对齐，并且可能除数为 0，所以需要加入枚举区间的右端点代表无穷特判

```
1 auto get_fl_r = [](int n, int l, int R_inf) -> int {
2     return l > n ? R_inf : min(R_inf, n / (n / l));
3 };
4 auto get_ce_r = [](int n, int l, int R_inf) -> int {
5     return l >= n ? R_inf : min(R_inf, (n - 1) / ((n - 1) / l));
6 };
7
8 for (int l = 1, r, R_inf = N; l <= R_inf; l = r + 1) { // 多维度记得循环条件取 max
9     r = min(get_fl_r(n1, l, R_inf), get_ce_r(n2, l, R_inf)); // 多维度记得取 min
10    ans += (f(r) - f(l - 1)) * g(n1 / l) * g((n2 + 1 - 1) / l);
11 }
```

应用： 转化为取整形式

- 积木大赛：** n 个数字 h_i ，每次可选区间 $\forall j \in [l, r], h_j > 0$ ，将 $h_j \leftarrow k$ ，求 $\forall k \in [1, \max h_i]$ ，使得 $\forall h_i \leq 0$ 的最小操作数
 - 看成若干段连续上升序列，令 $a_0 = 0$ ， $\forall k$ ，答案为 $\sum_{i=1}^n \max(0, \lceil \frac{h_i}{k} \rceil - \lceil \frac{h_{i-1}}{k} \rceil)$
 - 枚举 i 对 $k \in [1, \max h_i]$ 使用二维数论分块 $\{h_i, h_{i-1}\}$ 加上静态差分处理对每个 k 的贡献，时间复杂度 $O(n\sqrt{\max h_i})$
- $k \bmod i \Leftrightarrow k - \lfloor \frac{k}{i} \rfloor i$
- 结合莫反

欧拉函数&欧拉定理

欧拉函数： $\varphi(n)$ ，表示 $< n$ 和 n 互质的数的个数，当 n 为质数时， $\varphi(n) = n - 1$

- 性质：**
 - $\varphi(n)$ 为积性函数， $\gcd(a, b) = 1 \Rightarrow \varphi(a \times b) = \varphi(a) \times \varphi(b)$ （特别地， n 为奇数时， $\varphi(2n) = \varphi(n)$ ）
 - 欧拉反演：** $n = \sum_{d|n} \varphi(d)$
 - 若 $n = p^k, p$ 为质数 $\Rightarrow \varphi(n) = p^k - p^{k-1}$ (由定义得)
 - 由唯一分解定理，设 $n = \prod_{i=1}^s p_i^{k_i}, p_i$ 为质数 $\Rightarrow \varphi(n) = n \prod_{i=1}^s \frac{p_i - 1}{p_i}$

若 $k \bmod p = 0, p$ 为素数 $\Rightarrow \varphi(k \times p) = \varphi(k) \times p$
 - $\forall m, n \in \mathbb{Z}, nm \neq 0 \Rightarrow \varphi(mn)\varphi(\gcd(m, n)) = \varphi(m)\varphi(n)\gcd(m, n)$
- 实现：**

1. 只求一个数的欧拉函数,用质因数分解即可,可用Pollard Phi算法优化, 已经完成质因数分解时时间复杂度 $O(\log n)$

```
1 auto get_euler = [](int n, vector<array<int, 2>> & pri_fac) {
2     int ans = n;
3     for (auto & [p, cnt] : pri_fac) {
4         ans = ans / p * (p - 1);
5     }
6     return ans;
7 };
```

2. 求多个数的欧拉函数,用欧拉筛(线性筛)来求,时间复杂度 $O(n)$

```
1 vector<bool> not_prime; // 可以改为 hash_table 然后结合区间筛
2 vector<int> prime, phi;
3
4 auto Euler_sieve = [](int n) {
5     not_prime = vector<bool>(n + 1), prime = vector<int>(), phi = vector<int>(n + 1);
6     not_prime[1] = not_prime[0] = phi[1] = true; // true 为非质数
7     for (int i = 2; i <= n; i++) {
8         if (!not_prime[i]) {
9             prime.push_back(i);
10            phi[i] = i - 1; // 为素数时
11        } // 以下不是素数也需要筛
12        for (auto & x : prime) {
13            if (i * x > n) break;
14            not_prime[i * x] = true;
15            if (i % x == 0) {
16                phi[i * x] = phi[i] * x; //由唯一分解定理
17                break;
18            }
19            phi[i * x] = phi[i] * phi[x]; //积性函数性质
20        }
21    }
22 };
```

欧拉定理: 若 $\gcd(a, m) = 1 \Rightarrow a^{\varphi(m)} = 1 \pmod{m}$, 当 m 为质数时, $a^{m-1} = 1 \pmod{m}$ (费马小定理)

- (i) 求 $10^x \equiv 1 \pmod{m}$ 的最小正整数解, $\gcd(a, m) \neq 1 \Leftrightarrow$ 无解, 若有解 $\Rightarrow$ 必有 $x \mid \varphi(m)$ (反证)

扩展欧拉定理:
$$a^b \equiv \begin{cases} a^{b \bmod \varphi(m)}, & \gcd(a, m) = 1, \\ a^b, & \gcd(a, m) \neq 1, b < \varphi(m), \\ a^{(b \bmod \varphi(m)) + \varphi(m)}, & \gcd(a, m) \neq 1, b \geq \varphi(m). \end{cases} \pmod{m}$$

筛法

1. 埃拉托斯特尼筛法: 可筛至平方根,复杂度 $O(n \log \log n)$

```
1 vector<bool> not_prime; // 可以改为 hash_table 然后结合区间筛
2 vector<int> prime;
3
4 auto E_sieve = [](int n) {
5     not_prime = vector<bool>(n + 1), prime = vector<int>();
6     not_prime[1] = not_prime[0] = true; // true 为非质数
7     for (int i = 2; i <= n; i++) {
8         if (!not_prime[i]) { // 是质数
9             prime.push_back(i);
10            for (int j = i * i; j <= n; j += i) { //从i * i开始, 因为从2到i-1的倍数之前都筛过
11                not_prime[j] = true;
12            }
13        }
14    }
15 };
```

2. 欧拉筛(线性筛): 时间复杂度 $O(n)$, 可以得到每个数的最小质因子

```

1  auto [prime, mn_pri] = [](int n, vector<int> prime = {}) {
2      vector<int> mn_pri(n);
3      for (int i = 2; i < n; i++) {
4          if (!mn_pri[i]) prime.emplace_back(i), mn_pri[i] = i;
5          for (auto & x : prime) {
6              if (i * x >= n) break;
7              if (mn_pri[i * x] = x, i % x == 0) break;
8          }
9      }
10     return tuple{prime, mn_pri};
11 } (1e6 + 1);

```

筛区间素数： 使用 $\forall p \leq \sqrt{n}$ 可筛出 $1-n$ 的素数，时间复杂度 $O(n \ln n + \frac{\sqrt{r}}{\ln r})$, $n = r - l$

```

1  auto range_sieve = [](int l, int r) { // 需将普通筛的 not_prime 改为 hash_table
2      for (auto & p : prime) {
3          if (p * p > r) break;
4          for (int j = max((l + p - 1) / p, p) * p; j <= r; j += p) {
5              not_prime[j] = true;
6          }
7      }
8  };

```

筛法求约数个数： 用 $d(i)$ 表示 i 的约数个数， $num(i)$ 表示 i 的最小质因子出现次数

```

1  vector<bool> not_prime; // 可以改为 hash_table 然后结合区间筛
2  vector<int> prime, d, num;
3
4  auto Euler_sieve = [](int n) {
5      not_prime = vector<bool>(n + 1), prime = vector<int>(), d = num = vector<int>(n + 1);
6      not_prime[1] = not_prime[0] = d[1] = true; // true 为非质数
7      for (int i = 2; i <= n; i++) {
8          if (!not_prime[i]) {
9              prime.push_back(i);
10             num[i] = 1, d[i] = 2; // 为素数时
11         } // 以下不是素数也需要筛
12         for (auto & x : prime) {
13             if (i * x > n) break;
14             not_prime[x * i] = true;
15             if (i % x == 0) {
16                 num[i * x] = num[i] + 1, d[i * x] = d[i] / num[i * x] * (num[i * x] + 1); // 当x为i的素因数时
17                 break;
18             }
19             num[i * x] = 1, d[i * x] = d[i] << 1; // x与i互质时
20         }
21     }
22 };

```

筛法求约数和： f_i 表示 i 的约数和， g_i 表示 i 的最小质因子的 $p^0 + p^1 + p^2 + \dots + p^k$

```

1  vector<bool> not_prime; // 可以改为 hash_table 然后结合区间筛
2  vector<int> prime, f, g;
3
4  auto Euler_sieve = [](int n) {
5      not_prime = vector<bool>(n + 1), prime = vector<int>(), f = g = vector<int>(n + 1);
6      not_prime[1] = not_prime[0] = f[1] = g[1] = true; // true 为非质数
7      for (int i = 2; i <= n; i++) {
8          if (!not_prime[i]) {
9              prime.push_back(i);
10             g[i] = f[i] = i + 1; // 为素数时
11         } // 以下不是素数也需要筛
12         for (auto & x : prime) {
13             if (i * x > n) break;
14             not_prime[x * i] = true;
15             if (i % x == 0) {
16                 g[i * x] = g[i] * x + 1, f[i * x] = f[i] / g[i] * g[i * x]; // 当 x 为 i 的素因数时
17                 break;
18             }
19             g[i * x] = x + 1, f[i * x] = f[i] * f[x]; // x 与 i 互质时
20         }
21     }
22 };

```

筛一般积性函数： $f(i)$ 为积性函数，若 $\forall p \in \text{primes}, k \in \mathbb{N}$, 能 $O(1)$ 计算 $f(p^k)$ ，则能 $O(n)$ 筛出 $f(1) \sim f(n)$

- 设合数 $n = \prod_{i=1}^k p_i^{r_i}$, 其中 $p_1 < p_2 < \dots < p_k$, 令 $g(n) = p_1^{r_1}$, 若 n 被质数 p 筛掉 $x = \frac{n}{p} \Rightarrow g(n) = \begin{cases} pg(x), & x \bmod p = 0 \\ p, & otherwise \end{cases}$
- 若 $n = g(n)$, 则 $f(n) = f(g(n)) = f(p_1^{r_1})$, 否则 $f(n) = f(g(n)) \times f(\frac{n}{g(n)})$

分解质因数&约数倍数

约数个数定理: 若 $n = \prod_{i=1}^m p_i^{r_i} \Rightarrow d(n) = \prod_{i=1}^m (r_i + 1)$

分解质因数时间复杂度 $O(\sqrt{n})$, 若打出素数表为 $O(\sqrt{\frac{n}{\ln n}})$

```

1  auto Prime_Factor = [](int n, vector<array<int, 2>> pri_fac = {}) { // 0 为质因数, 1 为幂次
2      for (auto & p : prime) { // 打出素数表
3          if (p * p > n) break;
4          if (n % p == 0) {
5              pri_fac.push_back({p, 0});
6              while (n % p == 0) n /= p, pri_fac.back()[1]++;
7          }
8      }
9      if (n != 1) pri_fac.push_back({n, 1});
10     return move(pri_fac);
11 };
12 auto Prime_Factor = [](int n, vector<array<int, 2>> pri_fac = {}) { // 0 为质因数, 1 为幂次
13     for (int i = 2; i * i <= n; i++) {
14         if (n % i == 0) {
15             pri_fac.push_back({i, 0});
16             while (n % i == 0) n /= i, pri_fac.back()[1]++;
17         }
18     }
19     if (n != 1) pri_fac.push_back({n, 1}); // 留下一个素数
20     return move(pri_fac);
21 };

```

Pollard-Rho算法: 查找一个非平方因子时间复杂度 $O(n^{1/4})$

```

1  typedef long long ll; // 记得全部使用 long long (ll) 和快速乘
2
3  mt19937_64 rd(time(0));
4
5  auto binmul = [](ll a, ll b, ll mod) {
6      unsigned long long res = (unsigned long long)a * b - (unsigned long long)((long double)a / mod * b + 0.5L) *
mod;
7      return res < mod ? res : (res + mod) % mod;
8  };
9  auto binpow = [](ll a, ll b, ll mod, ll res = 1) {
10     for (a %= mod; b; b >>= 1, a = binmul(a, a, mod)) {
11         if (b & 1) {
12             res = binmul(res, a, mod);
13         }
14     }
15     return res;
16 };
17
18 auto MillerRabin = [](ll n) { // 基底测试
19     if (n < 4 || n % 2 == 0 || n % 3 == 0) return n == 2 || n == 3;
20     ll t = __builtin_ctzll(n - 1), u = (n - 1) >> t;
21     for (auto chk_num : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) { // 若为 int , 使用 {2, 7, 61}
22         ll x = chk_num % n, v = binpow(x, u, n), s = 0;
23         if (x <= 1 || x == n - 1 || v == 1) continue; // 通过此次测试
24         for (; s < t; s++, v = binmul(v, v, n)) {
25             if (v == n - 1) break; // 得到平凡平方根 n - 1
26         }
27         if (s == t) return false; // 找到非平凡平方根或无法通过 Fermat 素数测试
28     }
29     return true;
30 };
31
32 auto Pollard_Rho = [](ll n) {
33     ll c = rd() % (n - 1) + 1;
34     for (ll goal = 1, val = 1, s = 0, t = 0; ; goal <= 1, s = t, val = 1) {
35         for (ll step = 1; step <= goal; step++) {
36             (t = binmul(t, t, n) + c) %= n;

```

```

37         val = binmul(val, abs(t - s), n);
38         if (val == 0) return n; // 分解失败, 需重新分解
39         if (step % 127 == 0) { // 设置了块长为  $2^7 - 1$ 
40             ll d = gcd(val, n);
41             if (d > 1) return d;
42         }
43     }
44     ll d = gcd(val, n);
45     if (d > 1) return d;
46 }
47 };
48
49 auto Prime_Factor = [](ll n, vector<array<ll, 2>> pri_fac = {}) { // 0 为质因数, 1 为幂次
50     auto find = [&](auto && self, ll x, ll p = 4e18) -> void {
51         if (x < 2) return;
52         if (MillerRabin(x)) {
53             if (n % x == 0) { // 防止重复的质因子
54                 pri_fac.push_back({x, 0});
55                 while (n % x == 0) n /= x, pri_fac.back()[1]++;
56             }
57             return;
58         }
59         while (p >= x) p = Pollard_Rho(x);
60         while (x % p == 0) x /= p;
61         self(self, x), self(self, p);
62     };
63     find(find, n);
64     sort(pri_fac.begin(), pri_fac.end()); // 排序, 可以不使用, 这里使用复杂度影响不大
65     return move(pri_fac);
66 };

```

枚举所有因数：只求奇因数可用 `x >>= __builtin_ctzll(x)` 将其化为奇数

- 常见数据范围约数个数约为 $2 \times n^{\frac{1}{3}}$, 在 `int` 范围内最大为 1344, 在 `long long` 范围内最大为 103680
- $[1, n]$ 约数个数为由调和级数知为 $O(n \log n)$, 故随机数据下每个数期望约数个数为 $O(\log n)$

1. **根号枚举：**时间复杂度 $O(\sqrt{n})$

```

1 auto Get_Factor = [](int n, vector<int> fac = {}) {
2     for (int i = 1; i * i <= n; i++) {
3         if (n % i == 0) {
4             fac.push_back(i);
5             if (i * i != n) {
6                 fac.push_back(n / i);
7             }
8         }
9     }
10    // sort(fac.begin(), fac.end()); // 排序, 可以不使用, 这里使用对复杂度影响不大
11    return move(fac);
12 };

```

2. **质因数枚举：**先质因数分解后使用 *dfs* 枚举, 时间复杂度为 $O(d(n))$, $d(n)$ 为 n 的约数个数

```

1 auto Get_Factor = [](vector<array<int, 2>> & pri_fac, vector<int> fac = {}) {
2     for (auto & [p, cnt] : pri_fac) {
3         for (int i = 0, t = fac.size(); i < t; i++) { // 实测这样枚举最优
4             for (int j = 0, t = p; j < cnt; j++, t *= p) {
5                 fac.push_back(fac[i] * t);
6             }
7         }
8     }
9     // sort(fac.begin(), fac.end()); // 排序, 可以不使用, 这里使用对复杂度影响较大
10    return move(fac);
11 };

```

3. **枚举倍数求约数：**求出 $[1, n]$ 所有数的约数, 时间复杂度 $O(\sum_{i=1}^n d(i)) = O(n \log n)$

```

1 auto Get_Factor = [](int n, vector<vector<int>> fac = {}) {
2     for (int i = 1; i <= n; i++) {
3         for (int j = i; j <= n; j += i) {
4             fac[i].push_back(j);
5         }
6     }
7     return move(fac);
8 };

```

裴蜀定理

裴蜀定理： $\forall a, b \in \mathbb{Z}, ab \neq 0, \forall x, y \in \mathbb{Z} \Rightarrow \gcd(a, b) \mid ax + by \wedge \exists x, y \in \mathbb{Z}, s. t. ax + by = \gcd(a, b)$

推广：

- 1. **逆定理：** $\forall a, b \in \mathbb{Z}, ab \neq 0, d > 0 \mid \gcd(a, b), \exists x, y \in \mathbb{Z}, s. t. ax + by = d \Rightarrow d = \gcd(a, b)$
- 2. **多个整数：** 多个整数不全为 0 的线性组合也成立

推论： $\forall a, b, x, y \in \mathbb{N}, (a, b) = 1 \Rightarrow$ 记 $C = ab - a - b$ ，由 $(a, b) = 1$ ， C 必然为奇数，有 $\forall n \in \mathbb{Z}, n$ 与 $C - n$ 有且仅有一个能被 a, b 的线性组合表示，即 a, b 的线性组合表示的数在区间 $[0, C]$ 对称，0 可以被表示， C 不能被表示，负数不能被表示，大于 C 的可被表示

- **几何意义：** 将 $ax + by = n$ 视作直线，与两坐标轴在第一象限围成三角形，当 $n < ab$ 时这个直线在第一象限至多只能通过一个整点
 $\Leftrightarrow$ 作三角形 $(0, 0), (b, 0), (0, a)$ 随着 n 从 0 增大，直线右上方平移，整点会一个一个通过直线，直到最后撞上俩个整点
 $\Rightarrow \leq n$ 的能被表示的非负整数数量为直线 $ax + by = n$ （含）与两坐标轴（含）在第一象限覆盖的整点个数

应用：

- 1. 一张无限的整数点纸带，有 $n \in [1, 300]$ 张卡片，可以花 c_i 购买第 i 张卡片后从此可以从 x 跳转 $x - l_i, x + l_i$ 要求最小花费跳转纸带所有位置，不行输出 -1
 - 有解等价与选出的卡片的 l_i 的最大公约数为 1，使用 `unordered_map` 背包即可，由于 n 较小，可达 $\gcd$ 比较稀疏，时间复杂度 $O(nk)$ ，其中 k 为可达 $\gcd$ 个数

线性同余方程&乘法逆元

线性同余方程组： $ax \equiv n \pmod{b}$ 为线性同余方程,要从区间 $[0, b - 1]$ 求出 x ,当解不唯一时需要求出全体解

用乘法逆元求解： 在 $[0, b - 1]$ 有 $\gcd(a, b)$ 个整数解或 0 个整数解

- (i) 当 $\gcd(a, b) = 1$, 唯一解为 $x \equiv na^{-1} \pmod{b}$
- (ii) 当 $\gcd(a, b) \neq 1, \gcd(a, b) \nmid n \Leftrightarrow$ 无解
- (iii) 当 $\gcd(a, b) \neq 1, \gcd(a, b) \mid n \Leftrightarrow$ 有 $\gcd(a, b)$ 个解

a, b, n 均除以 $\gcd(a, b)$ 得 a', b', n' 有 $a'x \equiv n' \pmod{b'}$ 解得 x'
从而有 $x_i \equiv (x' + i \cdot b') \pmod{b}$ for $i = 0 \dots \gcd(a, b) - 1$

用exgcd求解： 有整数解 $\Leftrightarrow$ 在 $[0, b - 1]$ 有 $\gcd(a, b) \mid n$, 且解个数为 $\gcd(a, b)$

- (i) $ax \equiv n \pmod{b} \Leftrightarrow$ 不定式方程 $ax + by = n$

用 `exgcd` 计算 x_0, y_0 即 $ax_0 + by_0 = \gcd(a, b) \Leftrightarrow a \frac{n}{\gcd(a, b)} x_0 + b \frac{n}{\gcd(a, b)} y_0 = n$
 $\Rightarrow ax + by = n$ 的一组解为 $x = \frac{nx_0}{\gcd(a, b)}, y = \frac{ny_0}{\gcd(a, b)}$

- (ii) x_1, y_1 为方程 $ax + by = n$ 一组解 $\Rightarrow$ 任意解为 $\begin{cases} x = x_0 + \frac{bt}{\gcd(a, b)} \\ y = y_0 - \frac{at}{\gcd(a, b)} \end{cases}, t \in \mathbb{Z}$

故一般要求出最小整数解,即特解 $x = (x \bmod t + t) \bmod t$, 其中 $t = \frac{b}{\gcd(a, b)}$

时间复杂度均为 $O(\log n)$ ，若 $b = 0$ 交换 a, b

```
1 auto exgcd = [](int a, int b, array<int, 4> x = {1, 0, 0, 1}) -> array<int, 3> {
2     for (int c; b; tie(a, b) = pair{b, a % b}) {
3         c = a / b, x = {x[2], x[3], x[0] - x[2] * c, x[1] - x[3] * c};
4     }
5     return {x[0], x[1], a};
6 };
7
8 auto LiEu = [](int a, int b, int n = 1, int t = 0) -> array<int, 3> { // n = 1 求a 在 mod b 下逆元
9     auto [x, y, d] = exgcd(a, b); // 使用时需保证 b != 0, 枚举解时注意 a = 0
10    if (n % d) return {x, y, 0}; // 无解, 需要 gcd 时直接 ax + by
11    t = abs(b) / d, x = ((n / d * x) % t + t) % t; // 需要保证 t > 0
12    return {x, (n - a * x) / b, d}; // 当 n = 1 时 LiEu[0] 为逆元
13 };
```

乘法逆元：

定义： $ax \equiv 1 \pmod{b}$, 记 x 为 $a \bmod b$ 的逆元, 记 a^{-1}

exgcd求解： 同上

快速幂法： 运用费马小定理, p 为素数, $\gcd(a, p) = 1, a^{p-1} \equiv 1 \pmod{p}$, 即 $a^p \equiv a \pmod{p}$

故 $ax \equiv 1 \pmod{b} \Leftrightarrow ax \equiv a^{p-1} \pmod{b} \Leftrightarrow x \equiv a^{-1} \equiv a^{p-2} \pmod{b}$

注意：需要先使 $\gcd(a, p) = 1$

- 若取模完为 0 , 直接用 `zero` 记数标记, `zero>0` 答案为 0 , 否则不变
- 逆元无法使用时可用质因数分解法计算除法

时间复杂度 $O(\log n)$

```
1 auto binpow = [](int a, int b = mod - 2, int res = 1) { // 模数为质数, 求逆元, 可以对次幂 mod (mod - 1) 加速
2     for (a %= mod, b %= (mod - 1); b >= 1, (a *= a) %= mod) {
3         if (b & 1) {
4             (res *= a) %= mod;
5         }
6     }
7     return res;
8 };
```

线性求逆元:

1. 求 $[1, n]$ 的逆元, 由 $i^{-1} \equiv \begin{cases} 1, & i = 1 \\ \lfloor \frac{p}{i} \rfloor (p \bmod i)^{-1}, & \text{otherwise} \end{cases} \pmod{p}$ 时间复杂度 $O(n)$, 代码见 [排列组合](#)
2. 求任意 n 个数的逆元, 求出 n 前缀积 s_n , 再求出其逆元 sv_n , 则 $sv_{i-1} = sv_i \times a_i, i^{-1} = s_{i-1} \times sv_i \pmod{p}$ 时间复杂度 $O(n + \log p)$

```
1 vector<int> inv, fact, invfact;
2
3 auto init_inv = [](int n) {
4     inv = fact = invfact = vector<int>(n + 1, 1); // 注意 0 的 inv 无意义
5     for (int i = 1; i <= n; i++) { // 注意从 1 开始
6         (fact[i] = fact[i - 1] * i) %= mod;
7     }
8     invfact[n] = binpow(fact[n]);
9     for (int i = n; i > 0; i--) { // 注意到 1 结束
10        (invfact[i - 1] = invfact[i] * i) %= mod, (inv[i] = invfact[i] * fact[i - 1]) %= mod;
11    }
12 };
```

中国剩余定理

CRT: 求 $\forall i, j \in \mathbb{N}, 1 \leq i < j \leq k, \gcd(n_i, n_j) = 1$ 的线性同余方程
$$\begin{cases} x \equiv a_1 \pmod{n_1} \\ x \equiv a_2 \pmod{n_2} \\ \vdots \\ x \equiv a_k \pmod{n_k} \end{cases}$$
 的解

- 令 $N = \prod_{i=1}^k n_i, \forall i \in [1, k],$ 令 $m_i = \frac{N}{n_i}, m_i$ 在 $\bmod n_i$ 下的逆元 m_i^{-1} , 令 $c_i = m_i m_i^{-1}$ (不要 $\bmod n_i$) $\Rightarrow$ 唯一解
$$x = \sum_{i=1}^k a_i c_i \pmod{N}$$

```
1 auto CRT = [](vector<int> & a, vector<int> & n, int k, int N = 1, int x = 0) {
2     for (int i = 1; i <= k; i++) N = N * n[i]; // 使用时要保证全部值不会溢出, 特别是 N, x 和 m
3     for (int i = 1, m; i <= k; i++) {
4         m = N / n[i], (x += a[i] % n[i] * LiEu(m, n[i])[0] % N * m) %= N;
5     }
6     return (x % N + N) % N;
7 };
```

exCRT: 当模数不两两互质时, 求线性同余方程组的解

1. $\forall \begin{cases} x \equiv a_1 \pmod{n_1} \\ x \equiv a_2 \pmod{n_2} \end{cases} \Leftrightarrow \exists q_1, q_2 \in \mathbb{Z}, s.t. \begin{cases} q_1 n_1 + a_1 = x \\ q_2 n_2 + a_2 = x \end{cases} \Leftrightarrow q_1 n_1 - q_2 n_2 = a_2 - a_1$
2. 得通解 $\begin{cases} q_1' = q_1 + \frac{n_2 t}{\gcd(n_1, n_2)} \\ q_2' = q_2 + \frac{n_1 t}{\gcd(n_1, n_2)} \end{cases} \Rightarrow x_0 = q_1 n_1 + a_1 \Rightarrow$ 通解 $x = q_1' n_1 + a_1 = q_1 n_1 + a_1 + lcm(n_1, n_2) = x_0 + lcm(n_1, n_2)$
3. 即通解 $x = x_0 + lcm(n_1, n_2) \Leftrightarrow x \equiv x_0 \pmod{lcm(n_1, n_2)}$
4. 令 $a_0 = x_0, n_0 = lcm(n_1, n_2)$ 有 $x \equiv a_0 \pmod{n_0}$, 作为新的线性同余方程与下一个同余方程计算得解

```

1 auto exCRT = [](vector<int> & a, vector<int> & n, int k, int N = 1, int x = 0) -> int {
2     for (int i = 1; i <= k; i++) { // 注意是否溢出
3         auto [q, _, d] = LiEu(n[i], N, x - a[i]);
4         if (!d) return -1; // 不整除无解
5         N = lcm(N, n[i]), (x = q * n[i] + a[i]) %= N;
6     }
7     return (x % N + N) % N;
8 };

```

二次剩余

二次剩余: $n, p \in \mathbb{N}, (n, p) = 1, \exists x \in \mathbb{N}, \text{s.t. } x^2 \equiv n \pmod{p}$ 称 n 为 p 的二次剩余, 否则为二次非剩余

Euler判别法 p 为奇素数, $n \in \mathbb{N}, (n, p) = 1, n^{\frac{p-1}{2}} \equiv \begin{cases} 1, & \exists x \in \mathbb{Z}, \text{s.t. } x^2 \equiv n \pmod{p} \\ -1, & \text{otherwise} \end{cases} \pmod{p}$

二次剩余的数量: p 为奇素数, 模 p 意义下二次剩余和非二次剩余的数量均为 $\frac{p-1}{2}$

Cipolla算法: 求模奇素数下的二次剩余, 期望时间复杂度 $O(\log p)$

```

1 auto Cipolla = [](int n, int mod) -> set<int> { // size() 为 0 无解, 为 1 相同解, 为 2 两个解
2     if (!(n %= mod) || mod == 2) return {mod == 2};
3     if (binpow(n, mod - 1 >> 1, mod) == mod - 1) return {};
4     while (true) {
5         int a = rd() % mod, w = (a * a - n + mod) % mod;
6         if (binpow(w, mod - 1 >> 1, mod) == mod - 1) {
7             return [&](array<int, 2> a, int b, array<int, 2> res = {1, 0}) {
8                 auto mul = [&](auto & a, auto & b) {
9                     a = {(a[0] * b[0] + a[1] * b[1] % mod * w) % mod, (a[0] * b[1] + a[1] * b[0]) % mod};
10                };
11                for ( ; b >= 1, mul(a, a)) {
12                    if (b & 1) mul(res, a);
13                }
14                return res[0] << 1 == mod ? set{res[0]} : set{res[0], mod - res[0]};
15            } ({a, 1}, mod + 1 >> 1);
16        }
17    }
18 };

```

Tonelli-Shanks算法: 求模奇素数下的二次剩余, 时间复杂度 $O(\log^2 p)$

```

1 auto Tonelli_Shanks = [](int n, int p, int x = 0) -> set<int> {
2     if ((n %= p) == 0) return {0};
3     if (p == 2) return {1};
4     if (binpow(n, p >> 1, p) != 1) return {};
5     if (p & 2) {
6         x = binpow(n, p + 1 >> 2, p);
7         return x << 1 == p ? set{x} : set{x, p - x};
8     }
9     int s = __builtin_ctzll(p - 1);
10    int q = p >> s, z = 2;
11    for ( ; binpow(z, p >> 1, p) == 1; z++);
12    int c = binpow(z, q, p), r = binpow(n, q + 1 >> 1, p), t = binpow(n, q, p), tmp;
13    for (int m = s, i; t != 1; ) {
14        for (i = 0, tmp = t; tmp != 1; i++) {
15            (tmp *= tmp) %= p;
16        }
17        for ( ; i < --m; ) {
18            (c *= c) %= p;
19        }
20        (r *= c) %= p;
21        (c *= c) %= p;
22        (t *= c) %= p;
23    }
24    if (r << 1 == p) {
25        return {r};
26    }
27    return {r, p - r};
28 };

```

离散对数

离散对数： 定义与对数类似，取有原根的 $m \in \mathbb{N}^+$ ，设其中一个原根 $g \forall a \in \mathbb{Z}, (a, m) = 1, \exists ! k \in [0, \varphi(m))$, s.t. $g^k \equiv a \pmod{m}$ ，则称 k 为 g 为底，模 m 的离散对数，记 $k = \text{ind}_g a$

性质： 设 g 为模 m 的原根， $(a, m) = (b, m) = 1 \Rightarrow$

- $\text{ind}_g(a, b) \equiv \text{ind}_g a + \text{ind}_g b \pmod{\varphi(m)} \Rightarrow \text{ind}_g a^n \equiv n \text{ind}_g a \pmod{\varphi(m)}$
- 若 g_1 也为模 m 的原根，则 $\text{ind}_g a \equiv \text{ind}_g a \cdot \text{ind}_g g_1 \pmod{\varphi(m)}$
- $\text{ind}_g a \equiv \text{ind}_g b \pmod{\varphi(m)} \Leftrightarrow a \equiv b \pmod{m}$

大步小步算法： 求解 $a^x \equiv b \pmod{m}$ ，求 $x \in [0, m)$ 的解，当 $(a, m) = 1$ 时时间复杂度 $O(\sqrt{m})$ ，否则 $O(\sqrt{m} + \log m)$

```
1 auto BSGS = [](int a, int b, int mod, unordered_map<int, int> mp = {}) -> int { // 传入的 a, mod > 0, b >= 0
2     if ((b % mod) == 1 || mod == 1) return 0; // 传入的 (a, mod) = 1
3     if ((a % mod) == 0) return b ? -4e18 : 1;
4     int B = sqrt(mod) + 1, now = 1;
5     for (int i = 0; i < B; i++, (now *= a) %= mod) {
6         mp[now * b % mod] = i;
7     }
8     for (int i = 1, tmp = now; i <= B; i++, (tmp *= now) %= mod) {
9         if (mp.count(tmp)) return i * B - mp[tmp];
10    }
11    return -4e18; // 表示无解
12 };
13
14 auto exBSGS = [](int a, int b, int mod) -> int { // 传入的 a, mod > 0, b >= 0
15     if ((b % mod) == 1 || mod == 1) return 0;
16     if ((a % mod) == 0) return b ? -4e18 : 1; // 返回值为负，但不一定为 -4e18
17     for (int k = 1, cnt = 0, d = mod; ; k /= d, mod /= d, (k *= a / d) %= mod, cnt++) {
18         d = gcd(a, mod);
19         if (k == b) return cnt;
20         if (d == 1) return cnt + BSGS(a, b * LiEu(k, mod)[0], mod);
21         if (b % d) break;
22     }
23     return -4e18;
24 };
```

求 k 次剩余 求 $\forall a, b \in \mathbb{Z}, p \in \mathbb{P}$ 求解 $x^a \equiv b \pmod{m}$ ，时间复杂度 $O(\sqrt{m})$

- 令 $x = g^c$ ，其中 g 为 m 的原根，原问题转化为 $(g^a)^c \equiv b \pmod{p}$ ，特解 $x_0 \equiv g^c$
- 找到所有解： $\forall i \in \mathbb{Z}, x \equiv g^{c+i \cdot \frac{\varphi(p)}{\gcd(a, \varphi(p))}} \pmod{p}$

```
1 auto primitive_root = [](int mod) -> int { // mod 为质数
2     auto pri_fac = Prime_Factor(mod - 1);
3     for (int res = 2, flag = 1; res <= mod; res++, flag = 1) {
4         for (auto & [p, cnt] : pri_fac) {
5             if (binpow(res, (mod - 1) / p, mod) == 1) {
6                 flag = 0;
7                 break;
8             }
9         }
10        if (flag) return res;
11    }
12    return -1;
13 };
14 int g = primitive_root(mod);
15 int c = BSGS(binpow(g, a, mod), b, mod);
16 if (c < 0) {
17     cout << "0\n";
18     return;
19 }
20 int delta = (mod - 1) / gcd(a, mod - 1);
21 vector<int> ans;
22 for (int i = c % delta; i < mod - 1; i += delta) {
23     ans.push_back(binpow(g, i, mod));
24 }
```

莫比乌斯反演

莫比乌斯函数：

$$\mu = \begin{cases} 1 & n = 1 \\ 0 & n \text{ 的一个质因子次数超过一次} \\ (-1)^k & k \text{ 为 } n \text{ 的本质不同质因子个数} \end{cases}$$

性质:

$$\sum_{d|n} \mu(d) = \begin{cases} 1 & n = 1 \\ 0 & n \neq 1 \end{cases} \Leftrightarrow \sum_{d|n} \mu(d) = \varepsilon(n), \mu * 1 = \varepsilon$$

$$\text{反演结论: } [\gcd(i, j) = 1] = \sum_{d|\gcd(i, j)} \mu(d)$$

莫比乌斯变换:

$$(i) \quad f(n) = \sum_{d|n} g(d) \Leftrightarrow g(n) = \sum_{d|n} \mu(d) f\left(\frac{n}{d}\right), f(n) \text{ 为 } g(n) \text{ 的莫比乌斯变换, } g(n) \text{ 为 } f(n) \text{ 的莫比乌斯反演}$$

$$f = g * 1 \Leftrightarrow g = \mu * f$$

$$\text{证明: } \sum_{d|n} \mu(d) f\left(\frac{n}{d}\right) = \sum_{d|n} \mu(d) \sum_{i|\frac{n}{d}} g(i) = \sum_{i|n} g(i) \sum_{d|\frac{n}{i}} \mu(d) = g(n)$$

$$\mu * f = \mu * g * 1 = (\mu * 1) * g = \varepsilon * g = g$$

$$g * 1 = \mu * f * 1 = \varepsilon * f = f$$

$$(ii) \quad f(n) = \sum_{n|d} g(d) \Leftrightarrow g(n) = \sum_{n|d} \mu\left(\frac{d}{n}\right) f(d)$$

$$\text{证明: } \sum_{n|d} \mu\left(\frac{d}{n}\right) f(d) = \sum_{n|d} \mu\left(\frac{d}{n}\right) \sum_{d|i} g(i) = \sum_{n|i} g(i) \sum_{d|\frac{i}{n}} \mu(d) = g(n)$$

结论:

- $d(ij) = \sum_{x|ij} 1 = \sum_{x|i} \sum_{y|j} [\gcd(i, j) = 1], d(x) \text{ 为 } x \text{ 的约数个数}$
- $\sum_{i=1}^n \lfloor \frac{n}{i} \rfloor = \sum_{i=1}^n d(i)$

筛mobius函数: $O(n)$

```
1  vector<bool> not_prime; // 可以改为 hash_table 然后结合区间筛
2  vector<int> prime, mu;
3
4  auto Euler_sieve = [](int n) {
5      not_prime = vector<bool>(n + 1), prime = vector<int>(), mu = vector<int>(n + 1);
6      not_prime[1] = not_prime[0] = mu[1] = true; // true 为非质数
7      for (int i = 2; i <= n; i++) {
8          if (!not_prime[i]) {
9              prime.push_back(i);
10             mu[i] = -1; // 为素数时
11         } // 以下不是素数也需要筛
12         for (auto & x : prime) {
13             if (i * x > n) break;
14             not_prime[i * x] = true;
15             if (i % x == 0) {
16                 mu[i * x] = 0; // 当x为i的素因数时
17                 break;
18             }
19             mu[i * x] = -mu[i]; // x与i互质时
20         }
21     }
22 };
```

问题形式:

$$\text{求值(多组数据)} \sum_{i=x}^n \sum_{j=y}^m [\gcd(i, j) = k] \quad (1 \leq T, x, y, n, m, k \leq 5 \times 10^4)$$

解法一:

$$\text{用容斥原理分成四块} \sum_{i=1}^n \sum_{j=1}^m [\gcd(i, j) = k]$$

$$\Rightarrow \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} \sum_{j=1}^{\lfloor \frac{m}{k} \rfloor} [\gcd(i, j) = 1]$$

$$\Rightarrow \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} \sum_{j=1}^{\lfloor \frac{m}{k} \rfloor} \sum_{d|\gcd(i,j)} \mu(d)$$

$$\text{先枚举 } d \mid \gcd(i, j) \Rightarrow \sum_{d=1} \mu(d) \sum_{i=1}^{\lfloor \frac{n}{k} \rfloor} [d \mid i] \sum_{j=1}^{\lfloor \frac{m}{k} \rfloor} [d \mid j]$$

$$\Rightarrow \sum_{d=1}^{\min(\lfloor \frac{n}{k} \rfloor, \lfloor \frac{m}{k} \rfloor)} \mu(d) \lfloor \frac{n}{kd} \rfloor \lfloor \frac{m}{kd} \rfloor$$

解法二:

$$\text{设 } f(t) = \sum_{i=1}^n \sum_{j=1}^m [\gcd(i, j) = t]$$

$$F(t) = \sum_{i=1}^n \sum_{j=1}^m [t \mid \gcd(i, j)] = \lfloor \frac{n}{t} \rfloor \lfloor \frac{m}{t} \rfloor$$

$$\Rightarrow F(t) = \sum_{t \mid d} f(d)$$

$$\Rightarrow f(t) = \sum_{t \mid d} F(d) \mu(\frac{d}{t}) = \sum_d F(dt) \mu(d) = \sum_d \mu(d) \lfloor \frac{n}{dt} \rfloor \lfloor \frac{m}{dt} \rfloor$$

用数论分块解决, 时间复杂度 $O(N + T\sqrt{n})$

```

1  void qia() {
2      int a, b, c, d, k;
3      cin >> a >> b >> c >> d >> k;
4
5      auto solve = [&](int b, int d) {
6          int res = 0;
7
8          for (int i = 1, j; i <= min(b, d); i = j + 1) {
9              j = min(b / (b / i), d / (d / i));
10             res += (sum_mu[j] - sum_mu[i - 1]) * (b / i) * (d / i);
11         }
12
13         return res;
14     };
15
16     cout << solve(b / k, d / k) - solve((a - 1) / k, d / k) - solve(b / k, (c - 1) / k) + solve((a - 1) / k, (c - 1) / k) << "\n";
17 }

```

- 与 **bitset** 结合: $1 \leq n \leq 10^5$ 个空可重集 A_i , 4 种操作 $1 \leq q \leq 10^6$:

1. 令 $A_i = \{x\}$, $1 \leq x \leq 7000$
2. 令 $A_i = A_j \cup A_k$
3. 令 $A_i = \{\gcd(x, y) \mid x \in A_j, y \in A_k\}$
4. 查 A_i 中 x 的数量 $\text{cnt}_x \bmod 2$

- 用 **bitset** 维护 A_i 所有约数构成的可重集 B_i

1. 预处理值域内每个数所有约数构成的可重集 **bitset** 即可
2. 直接异或
3. 按位与

$$4. \text{cnt}_x = \sum_{t \in A_i} [\frac{t}{x} = 1] = \sum_{t \in A_i} \sum_{d \mid \frac{t}{x}} \mu(d) = \sum_{t \in A_i} \sum_{d \mid t} \mu(d) = \sum_{dx \in B_i} \mu(d) = \sum_{d \in B_i, x \mid d} \mu(\frac{d}{x})$$

枚举所有 x 的倍数 d , 处理出满足 $\mu(\frac{d}{x}) \equiv 1 \pmod{2}$ 的 d 置 1 的 **bitset**, 按位与后 **count() % 2** 即可

- 时间复杂度 $O(\frac{qv}{w} + v \log v)$

威尔逊定理

$$\forall n, p \in N^+, (n!)_p \Leftrightarrow \prod_{i \in N^+, i \leq n \wedge p \nmid i} i$$

1. **Wilson 定理**: $\forall p \in \text{prime}, (p!)_p \equiv (p-1)! \equiv -1 \pmod{p}$

$$\Rightarrow (p-1)! \equiv \begin{cases} 2, & p=4 \\ -1, & p \in \text{prime} \\ 0, & \text{otherwise} \end{cases} \pmod{p}$$

2. Wilson 定理的推广:

Min_25筛

```

1  #include<cstdio>
2  #include<algorithm>
3  #include<cstring>
4  #include<cmath>
5  #define ll long long
6  using namespace std;
7  const ll MOD=1000000007, inv2=500000004, inv3=333333336;
8  ll prime[1000005], num, sp1[1000005], sp2[1000005];
9  ll n, Sqr, tot, g1[1000005], g2[1000005], w[1000005], ind1[1000005], ind2[1000005];
10 bool flag[1000005];
11 void pre(int n) //预处理, 线性筛
12 {
13     flag[1]=1;
14     for(int i=1; i<=n; i++)
15     {
16         if(!flag[i])
17         {
18             prime[++num]=i;
19             sp1[num]=(sp1[num-1]+i)%MOD;
20             sp2[num]=(sp2[num-1]+1ll*i*i)%MOD;
21         }
22         for(int j=1; j<=num&&prime[j]*i<=n; j++)
23         {
24             flag[i*prime[j]]=1;
25             if(i%prime[j]==0) break;
26         }
27     }
28 }
29 ll S(ll x, int y) //第二部分
30 {
31     if(prime[y]>=x) return 0;
32     ll k=x<=Sqr?ind1[x]:ind2[n/x];
33     ll ans=(g2[k]-g1[k]+MOD-(sp2[y]-sp1[y]+MOD)%MOD)%MOD;
34     for(int i=y+1; i<=num&&prime[i]*prime[i]<=x; i++)
35     {
36         ll pe=prime[i];
37         for(int e=1; pe<=x; e++, pe=pe*prime[i])
38         {
39             ll xx=pe%MOD;
40             ans=(ans+xx*(xx-1)%MOD*(S(x/pe, i)+(e!=1)))%MOD;
41         }
42     }
43     return ans%MOD;
44 }
45 int main()
46 {
47     scanf("%lld", &n);
48     Sqr=sqrt(n);
49     pre(Sqr);
50     for(ll i=1; i<=n; i++)
51     {
52         ll j=n/(n/i);
53         w[++tot]=n/i;
54         g1[tot]=w[tot]%MOD;
55         g2[tot]=g1[tot]*(g1[tot]+1)/2%MOD*(2*g1[tot]+1)%MOD*inv3%MOD;
56         g2[tot]--;
57         g1[tot]=g1[tot]*(g1[tot]+1)/2%MOD-1;
58         if(n/i<=Sqr) ind1[n/i]=tot;
59         else ind2[n/(n/i)]=tot;
60         i=j+1;
61     } //g1, g2分别表示一次项和二次项, ind1和ind2用来记录这个数在数组中的位置
62     for(int i=1; i<=num; i++) //由于g数组可以滚动, 所以就只开了一维
63     {
64         for(int j=1; j<=tot&&prime[i]*prime[i]<=w[j]; j++)
65         {
66             ll k=w[j]/prime[i]<=Sqr?ind1[w[j]/prime[i]]:ind2[n/(w[j]/prime[i])];

```

```

67         g1[j] -= prime[i] * (g1[k] - sp1[i-1] + MOD) % MOD;
68         g2[j] -= prime[i] * prime[i] % MOD * (g2[k] - sp2[i-1] + MOD) % MOD;
69         g1[j] %= MOD, g2[j] %= MOD;
70         if (g1[j] < 0) g1[j] += MOD;
71         if (g2[j] < 0) g2[j] += MOD;
72     }
73 }
74 printf("%lld\n", (S(n, 0) + 1) % MOD);
75 return 0;
76 }

```

多项式与生成函数

快速傅里叶变换

多项式牛顿迭代

多项式牛顿迭代：当 $g(f(x)) \equiv 0 \pmod{x^n}$ 求解 x^n 意义下的 $f(x)$

1. 当 $n = 1$, $[x^0]g(f(x)) = 0$ 的解需要单独求出
2. 考虑倍增, 设在模 $x^{\lceil \frac{n}{2} \rceil}$ 意义下的解为 $f_0(x)$, 求模 x^n 意义下的解 $f(x) \Rightarrow$
3. $f(x) \equiv f_0(x) - \frac{g(f_0(x))}{g'(f_0(x))} \pmod{x^n}$ 用主定理分析复杂度即可, 一般为 $O(n \log n)$

多项式多点求值|快速插值

多项式多点求值：给出多项式 $f(x)$ 和 n 个点 $x_1, x_2, \dots, x_n$, 求解 $f(x_1), f(x_2), \dots, f(x_n)$, 时间复杂度为 $O(n \log^2 n)$

Lagrange 插值：形式为: $f(x) = \sum_{i=1}^n y_i \cdot \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$ 时间复杂度为 $O(n \log^2 n)$

横坐标是连续整数的 Lagrange 插值：做到 $O(n)$ 插值去计算单点。

设要求 n 次多项式为 $f(x)$, 已知 $f(1), \dots, f(n+1) (1 \leq i \leq n+1) \Rightarrow f(x) = \sum_{i=1}^{n+1} y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j} = \sum_{i=1}^{n+1} y_i \prod_{j \neq i} \frac{x - j}{i - j}$

后面的累乘可以分子分母分别考虑, 得到分子为: $\frac{\prod_{j=1}^{n+1} (x - j)}{x - i}$

分母的 $i - j$ 累乘可以拆成两段阶乘来算: $(-1)^{n+1-i} \cdot (i-1)! \cdot (n+1-i)!$

于是横坐标为 $1, \dots, n+1$ 的插值公式:

$$f(x) = \sum_{i=1}^{n+1} (-1)^{n+1-i} y_i \cdot \frac{\prod_{j=1}^{n+1} (x - j)}{(i-1)!(n+1-i)!(x-i)}$$

预处理 $(x - i)$ 前后缀积、阶乘和阶乘逆, 然后代入这个式子, 复杂度为 $O(n)$ 。

```

1  constexpr int mod = 998244353, g[2] = {332748118, 3};
2  constexpr int MOD[] = {998244353, 1004535809, 754974721, 469762049}, G[][2] = {{332748118, 3}, {334845270, 3},
3  {617706590, 11}, {156587350, 3}};
4
5  auto binpow = [](int a, int b, int mod, int res = 1) { // 模数为质数, 求逆元, 可以对次幂 mod (mod - 1) 加速
6      for (a %= mod; b; b >>= 1, (a *= a) %= mod) {
7          if (b & 1) {
8              (res *= a) %= mod;
9          }
10     }
11     return res;
12 };
13
14 auto Cipolla = [](int n, int mod) -> set<int> { // size() 为 0 无解, 为 1 相同解, 为 2 两个解
15     if (!(n %= mod) || mod == 2) return {mod == 2};
16     if (binpow(n, mod - 1 >> 1, mod) == mod - 1) return {};
17     while (true) {
18         int a = rd() % mod, w = (a * a - n + mod) % mod;
19         if (binpow(w, mod - 1 >> 1, mod) == mod - 1) {

```

```

19         return [&](array<int, 2> a, int b, array<int, 2> res = {1, 0}) {
20             auto mul = [&](auto &a, auto &b) {
21                 a = {(a[0] * b[0] + a[1] * b[1] % mod * w) % mod, (a[0] * b[1] + a[1] * b[0]) % mod};
22             };
23             for ( ; b; b >>= 1, mul(a, a)) {
24                 if (b & 1) mul(res, a);
25             }
26             return res[0] << 1 == mod ? set{res[0]} : set{res[0], mod - res[0]};
27         } ({a, 1}, mod + 1 >> 1);
28     }
29 }
30 };
31
32 template <int mod = mod, const int *g = g>
33 struct Poly : vector<int> {
34     int deg;
35     static vector Inv;
36     static array<vector, 2> W;
37
38     Poly & Resize(int sz, int x = 0) { // !! 改度必须用这个改, 不能手动改, 别用错 Resize()
39         resize(sz, x), resize((deg = sz - 1) > 0 ? (1ll << 64 - __builtin_clzll(deg)) : 1, x);
40         for (int i = Inv.size(); i <= size(); i++) {
41             Inv.emplace_back((mod - mod / i) * Inv[mod % i] % mod);
42         }
43         for (int bit = W[0].size(); 1ll << bit < size(); bit++) {
44             for (auto op : {0, 1}) {
45                 W[op].emplace_back(::binpow(g[op], mod >> bit + 1, mod));
46             }
47         }
48         return *this;
49     } Poly(int sz = 1, int x = 0) {
50         Resize(sz, x);
51     } Poly(const Poly & f, int sz = 0, int x = 0) : vector(f), deg(f.deg) {
52         if (sz) Resize(sz, x); // 拷贝, 有要改 sz 才改
53     } Poly(vector::iterator l, vector::iterator r, int sz = 0, int x = 0) : vector(l, r) {
54         Resize(sz ? sz : max<int>(1, size()), x); // 要注意传入的 size 为 0 的情况
55     } Poly(initializer_list<int> f) : vector(f) {
56         Resize(max<int>(1, size()));
57     }
58
59     void NTT(int op) { // 1 为 DFT, 0 为 IDFT
60         vector<int> Rev(size());
61         for (int i = 0; i < size(); i++) {
62             Rev[i] = ((i & 1) * size() | Rev[i >> 1]) >> 1;
63             if (i < Rev[i]) ::swap(at(i), at(Rev[i]));
64         }
65         for (int h = 1, t = 0; h < size(); h <= 1, t++) {
66             for (int j = 0; j < size(); j += h < 1) {
67                 for (int k = j, tw = 1; k < j + h; k++, (tw *= W[op][t]) %= mod) {
68                     (at(k + h) = at(k) - tw * at(k + h) % mod + mod) %= mod;
69                     (at(k) += at(k) - at(k + h) + mod) %= mod;
70                 }
71             }
72         }
73         for (int i = 0; !op && i < size(); i++) {
74             (at(i) *= Inv[size()]) %= mod;
75         }
76     } Poly & operator *= (const Poly & t) {
77         Resize(deg + t.deg + 1);
78         Poly b(t, deg + 1);
79         NTT(1), b.NTT(1);
80         for (int i = 0; i < size(); i++) {
81             (at(i) *= b[i]) %= mod;
82         }
83         return NTT(0), *this;
84     } Poly operator * (const Poly & t) { return Poly(*this) *= t; }
85     Poly & operator *= (const int & k) {
86         for (int i = 0; i <= deg; i++) {
87             (at(i) *= k) %= mod;
88         }
89         return *this;
90     } Poly operator * (const int & k) { return Poly(*this) *= k; }
91
92     Poly & operator += (const Poly & t) {
93         Resize(max(deg, t.deg) + 1);

```

```

94     for (int i = 0; i <= t.deg; i++) {
95         (at(i) += t[i]) %= mod;
96     }
97     return *this;
98 } Poly operator + (const Poly & t) { return Poly(*this) += t; }
99 Poly & operator += (const Poly & t) {
100     Resize(max(deg, t.deg) + 1);
101     for (int i = 0; i <= t.deg; i++) {
102         (at(i) += mod - t[i]) %= mod;
103     }
104     return *this;
105 } Poly operator - (const Poly & t) { return Poly(*this) -= t; }
106
107 Poly & operator /= (const Poly & t) {
108     return *this = DivMod(t)[0];
109 } Poly operator / (const Poly & t) { return DivMod(t)[0]; }
110 Poly & operator %= (const Poly & t) {
111     return *this = DivMod(t)[1];
112 } Poly operator % (const Poly & t) { return DivMod(t)[1]; }
113
114 Poly & operator >= (const int & k) { // 消灭低次项
115     if (deg < k) return *this = Poly();
116     copy(begin() + k, begin() + deg + 1, begin());
117     return Resize(deg - k + 1);
118 } Poly operator >> (const int & k) { return Poly(*this) >>= k; }
119 Poly & operator <= (const int & k) { // 增加低次项
120     insert(begin(), k, 0);
121     return Resize(deg + k + 1);
122 } Poly operator << (const int & k) { return Poly(*this) <<= k; }
123
124 Poly inv() { // 存在当且仅当 at(0) != 0, 使用时用临时变量开至模 x^sz 大小
125     Poly f(1, binpow(at(0), mod - 2, mod)); // f 为逆
126     for (int bit = 2; bit >> 1 <= deg; bit <= 1) { // h 为原函数
127         Poly h(begin(), begin() + bit, bit <= 1); // 要开四倍空间
128         f.Resize(h.size()); // 俩个函数 (bit >> 1) 和 一个为 bit 相乘
129         f.NTT(1), h.NTT(1);
130         for (int i = 0; i < h.size(); i++) {
131             (f[i] *= 2 - f[i] * h[i] % mod + mod) %= mod;
132         }
133         f.NTT(0), f.Resize(bit);
134     }
135     return f.Resize(deg + 1); // 记得更改度
136 }
137
138 array<Poly, 2> DivMod(const Poly & b) {
139     if (deg < b.deg) return {Poly(), *this}; // 小于直接返回
140     Poly r(*this), q(b); // 要先取反再 Resize
141     reverse(r.begin(), r.begin() + r.deg + 1), reverse(q.begin(), q.begin() + q.deg + 1);
142     q = (q.Resize(deg - b.deg + 1).inv() * r).Resize(deg - b.deg + 1);
143     reverse(q.begin(), q.begin() + q.deg + 1);
144     return {q, (*this - q * b).Resize(b.deg)};
145 }
146
147 int GetDelta() { // at(0) == 0 的转化
148     return find_if(begin(), end(), [](auto & x) { return x != 0; }) - begin();
149 }
150
151 Poly EasySqrt(int delta = 0, int root0 = 1) { // 只用于 at(0) = 1, 使用时用临时变量开至模 x^n 大小
152     Poly f(1, root0);
153     for (int bit = 2; (bit >> 1) + delta <= deg; bit <= 1) {
154         Poly h(begin() + delta, begin() + bit + delta, bit <= 1), invf = f.Resize(bit).inv();
155         f.Resize(h.size()), invf.Resize(h.size());
156         f.NTT(1), h.NTT(1), invf.NTT(1);
157         for (int i = 0; i < h.size(); i++) {
158             (f[i] = Inv[2] * (f[i] + h[i] * invf[i] % mod) % mod) %= mod;
159         }
160         f.NTT(0), f.Resize(bit);
161     }
162     return f.Resize(deg + 1); // 记得更改度
163 } vector<Poly> sqrt(int delta = 0, vector<Poly> res = {}) { // 考虑所以情况和所有解, size为 0 表示无解
164     if ((delta = GetDelta()) > deg) return {*this}; // 全 0
165     if (delta & 1) return {}; // 无解
166     auto root = Cipolla(at(delta), mod); // 求字典序最小可以拿出来只跑一次
167     for (auto & x : root) {
168         res.push_back(EasySqrt(delta, x));

```

```

169     }
170     return res;
171 }
172
173 Poly dev() { // 求导
174     Poly f(deg);
175     for (int i = 1; i <= deg; i++) {
176         (f[i - 1] = i * at(i)) %= mod;
177     }
178     return f;
179 } Poly indev() { // 积分
180     Poly f(deg + 2);
181     for (int i = 0; i <= deg; i++) {
182         (f[i + 1] = Inv[i + 1] * at(i)) %= mod;
183     }
184     return f;
185 }
186
187 Poly ln() { // 存在当且仅当 at(0) = 1
188     return (dev() * inv()).Resize(deg).indev();
189 } Poly exp() { // 存在当且仅当 at(0) = 0
190     Poly f(1, 1);
191     for (int bit = 2; bit >> 1 <= deg; bit <= 1) {
192         Poly h(begin(), begin() + bit, bit << 1), lnf = f.Resize(bit).ln();
193         f.Resize(h.size()), lnf.Resize(h.size());
194         f.NTT(1), h.NTT(1), lnf.NTT(1);
195         for (int i = 0; i < h.size(); i++) {
196             (f[i] *= (1 - lnf[i] + mod + h[i])) %= mod;
197         }
198         f.NTT(0), f.Resize(bit);
199     }
200     return f.Resize(deg + 1);
201 }
202
203 Poly pow(string s, int delta = 0, int k0 = 0, int k1 = 0) { // 使用的时候 to_string
204     if ((delta = GetDelta()) > deg) return *this; // 全 0
205     for (auto & x : s) {
206         (k0 = k0 * 10 + (x - '0')) %= mod, (k1 = k1 * 10 + (x - '0')) %= mod - 1;
207         if (delta * k1 > deg) return Poly(deg + 1); // mod x^sz 后为 0
208     }
209     auto f = ((*this) >> delta).Resize(deg - delta * k1 + 1) * binpow(at(delta), mod - 2, mod);
210     return (f.ln() * k0).exp() * binpow(at(delta), k1, mod) << delta * k1;
211 }
212 // 使用的点都要互不相同
213 static Poly & DivFFT(iterator l, iterator r, vector<Poly> & tr, int op = 1, int p = 1) { // 求横坐标轴上插
    值, op = 1 正常, op = 0 求转置
214     if (next(l) == r) return op ? tr[p] = {mod - *l, 1} : tr[p] = {1, mod - *l}; // 左闭右开
215     auto mid = next(l, r - l + 1 >> 1);
216     return tr[p] = DivFFT(l, mid, tr, op, p << 1) * DivFFT(mid, r, tr, op, p << 1 | 1);
217 }
218 Poly T_Mul(Poly & b, Poly c = {}) { // 辅助函数, 转置乘法, 会修改 b
219     reverse(b.begin(), b.begin() + b.deg + 1), c = *this * b;
220     return Poly(c.begin() + b.deg, c.begin() + c.deg + 1);
221 }
222 vector MultiPoint(iterator L, iterator R, vector<int> res = {}) { // 多点求值, 左闭右开
223     vector<Poly> tr(max<int>(deg + 1, R - L) << 2); // 传入 n 个点, 返回 n 个对应点
224     Poly x(L, R, max<int>(deg + 1, R - L));
225     auto Find = [&](auto && self, auto l, auto r, int p = 1) { // 求值过程
226         if (next(l) == r) return res.push_back(tr[p][0]);
227         auto mid = next(l, r - l + 1 >> 1);
228         tr[p].Resize(r - l + 1);
229         tie(tr[p << 1], tr[p << 1 | 1]) = pair{tr[p].T_Mul(tr[p << 1 | 1]), tr[p].T_Mul(tr[p << 1])};
230         self(self, l, mid, p << 1), self(self, mid, r, p << 1 | 1);
231     };
232     tr[1] = T_Mul(tr[1] = DivFFT(x.begin(), x.begin() + x.deg + 1, tr, 0).inv());
233     return Find(Find, x.begin(), x.begin() + x.deg + 1, res.resize(R - L), res);
234 }
235 static Poly QuickPolate(vector<array<int, 2>>::iterator L, vector<array<int, 2>>::iterator R) {
236     vector<Poly> tr(R - L << 2); // 快速插值, 给二维点求函数, 左闭右开
237     Poly x(R - L);
238     for (auto i = L; i != R; i++) {
239         x[i - L] = i -> at(0);
240     }
241     auto ty = DivFFT(x.begin(), x.begin() + x.deg + 1, tr, 1).dev().MultiPoint(x.begin(), x.begin() + x.deg
    + 1);

```

```

242     auto Find = [&](auto && self, auto l, auto r, int p = 1) { // 求值过程
243         if (next(l) == r) return Poly(1, l -> at(1) * binpow(ty[l - L], mod - 2, mod) % mod);
244         auto mid = next(l, r - 1 + 1 >> 1);
245         return self(self, l, mid, p << 1) * tr[p << 1 | 1] + self(self, mid, r, p << 1 | 1) * tr[p << 1];
246     };
247     return Find(Find, L, R);
248 }
249 };
250
251 struct MTT : Poly<mod, g> {
252     int mod; // 实际的 mod
253
254     MTT & GetMod() {
255         for (int i = 0; i < size(); i++) {
256             at(i) %= mod;
257         }
258         return *this;
259     } MTT(int n, int mod = MOD[0], int x = 0) : mod(mod), Poly(n, x) {
260         GetMod();
261     } MTT(Poly<MOD[0], G[0]> & f, int mod = MOD[0]) : mod(mod), Poly(f.begin(), f.begin() + f.deg + 1) {
262         GetMod();
263     } MTT(Poly<MOD[1], G[1]> & f, int mod = MOD[0]) : mod(mod), Poly(f.begin(), f.begin() + f.deg + 1) {
264         GetMod();
265     } MTT(Poly<MOD[2], G[2]> & f, int mod = MOD[0]) : mod(mod), Poly(f.begin(), f.begin() + f.deg + 1) {
266         GetMod();
267     }
268     void merge(auto & a, auto & b, int id, int inv, int cmod) { // 中国剩余定理合并
269         for (int i = 0; i <= a.deg; i++) {
270             int K = (a[i] % MOD[id] + MOD[id] - b[i] % MOD[id]) * inv % MOD[id];
271             a[i] = K * cmod + b[i];
272         }
273     }
274     MTT operator * (MTT & b) {
275         Poly<MOD[0], G[0]> a0(begin(), end(), deg + 1), b0(b.begin(), b.end(), b.deg + 1);
276         Poly<MOD[1], G[1]> a1(begin(), end(), deg + 1), b1(b.begin(), b.end(), b.deg + 1);
277         Poly<MOD[2], G[2]> a2(begin(), end(), deg + 1), b2(b.begin(), b.end(), b.deg + 1);
278         a0 = a0 * b0, a1 = a1 * b1, a2 = a2 * b2;
279         merge(a1, a0, 1, 669690699, MOD[0]), merge(a2, a1, 2, 681249294, MOD[0] * MOD[1] % mod);
280         return MTT(a2, mod);
281     } MTT & operator *= (MTT & t) { return *this = *this * t; }
282 };
283
284 template <int mod, const int* G>
285 vector<int> Poly<mod, G>::Inv(2, 1);
286 template <int mod, const int* G>
287 array<vector<int>, 2> Poly<mod, G>::W;

```

```

1 // #pragma GCC optimize(2)
2 #include <bits/stdc++.h>
3 using namespace std;
4 typedef long long ll;
5
6 const int N = 3000007;
7 const int p = 998244353, gg = 3, ig = 332738118, img = 86583718;
8 const int mod = 998244353;
9
10 template <class T> void read(T &x)
11 {
12     x = 0;
13     register int f = 1;
14     register char ch = getchar();
15     while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
16     while(ch >= '0' && ch <= '9') {x = x * 10 + ch - '0'; ch = getchar();}
17     x *= f;
18 }
19
20 int qpow(int a, int b)
21 {
22     int res = 1;
23     while(b) {
24         if(b & 1) res = 1ll * res * a % mod;
25         a = 1ll * a * a % mod;
26         b >>= 1;
27     }
28     return res;

```

```

29     }
30
31     namespace Poly
32     {
33         #define mul(x, y) (1ll * x * y >= mod ? 1ll * x * y % mod : 1ll * x * y)
34         #define minus(x, y) (1ll * x - y < 0 ? 1ll * x - y + mod : 1ll * x - y)
35         #define plus(x, y) (1ll * x + y >= mod ? 1ll * x + y - mod : 1ll * x + y)
36         #define ck(x) (x >= mod ? x - mod : x) //取模运算太慢了
37
38         typedef vector<int> poly;
39         const int G = 3; //根据具体的模数而定, 原根可不一定不一样!!!
40         //一般模数的原根为 2 3 5 7 10 6
41         const int inv_G = qpow(G, mod - 2);
42         int RR[N], deer[2][19][N], inv[N];
43
44         void init(const int t) { //预处理出来NTT里需要的w和wn, 砍掉了一个log的时间
45             for(int p = 1; p <= t; ++ p) {
46                 int buf1 = qpow(G, (mod - 1) / (1 << p));
47                 int buf0 = qpow(inv_G, (mod - 1) / (1 << p));
48                 deer[0][p][0] = deer[1][p][0] = 1;
49                 for(int i = 1; i < (1 << p); ++ i) {
50                     deer[0][p][i] = 1ll * deer[0][p][i - 1] * buf0 % mod; //逆
51                     deer[1][p][i] = 1ll * deer[1][p][i - 1] * buf1 % mod;
52                 }
53             }
54             inv[1] = 1;
55             for(int i = 2; i <= (1 << t); ++ i)
56                 inv[i] = 1ll * inv[mod % i] * (mod - mod / i) % mod;
57         }
58
59         int NTT_init(int n) { //快速数论变换预处理
60             int limit = 1, L = 0;
61             while(limit <= n) limit <<= 1, L ++ ;
62             for(int i = 0; i < limit; ++ i)
63                 RR[i] = (RR[i >> 1] >> 1) | ((i & 1) << (L - 1));
64             return limit;
65         }
66
67         void NTT(poly &A, int type, int limit) { //快速数论变换
68             A.resize(limit);
69             for(int i = 0; i < limit; ++ i)
70                 if(i < RR[i])
71                     swap(A[i], A[RR[i]]);
72             for(int mid = 2, j = 1; mid <= limit; mid <<= 1, ++ j) {
73                 int len = mid >> 1;
74                 for(int pos = 0; pos < limit; pos += mid) {
75                     int *wn = deer[type][j];
76                     for(int i = pos; i < pos + len; ++ i, ++ wn) {
77                         int tmp = 1ll * (*wn) * A[i + len] % mod;
78                         A[i + len] = ck(A[i] - tmp + mod);
79                         A[i] = ck(A[i] + tmp);
80                     }
81                 }
82             }
83             if(type == 0) {
84                 for(int i = 0; i < limit; ++ i)
85                     A[i] = 1ll * A[i] * inv[limit] % mod;
86             }
87         }
88
89         poly poly_mul(poly A, poly B) { //多项式乘法
90             int deg = A.size() + B.size() - 1;
91             int limit = NTT_init(deg);
92             poly C(limit);
93             NTT(A, 1, limit);
94             NTT(B, 1, limit);
95             for(int i = 0; i < limit; ++ i)
96                 C[i] = 1ll * A[i] * B[i] % mod;
97             NTT(C, 0, limit);
98             C.resize(deg);
99             return C;
100         }
101
102         poly poly_inv(poly &f, int deg) { //多项式求逆
103             if(deg == 1)

```

```

104         return poly(1, qpow(f[0], mod - 2));
105
106     poly A(f.begin(), f.begin() + deg);
107     poly B = poly_inv(f, (deg + 1) >> 1);
108     int limit = NTT_init(deg << 1);
109     NTT(A, 1, limit), NTT(B, 1, limit);
110     for(int i = 0; i < limit; ++ i)
111         A[i] = B[i] * (2 - 111 * A[i] * B[i] % mod + mod) % mod;
112     NTT(A, 0, limit);
113     A.resize(deg);
114     return A;
115 }
116
117 poly poly_dev(poly f) { //多项式求导
118     int n = f.size();
119     for(int i = 1; i < n; ++ i) f[i - 1] = 111 * f[i] * i % mod;
120     return f.resize(n - 1), f; //f[0] = 0, 这里直接扔了, 从1开始
121 }
122
123 poly poly_idev(poly f) { //多项式求积分
124     int n = f.size();
125     for(int i = n - 1; i ; -- i) f[i] = 111 * f[i - 1] * inv[i] % mod;
126     return f[0] = 0, f;
127 }
128
129 poly poly_ln(poly f, int deg) { //多项式求对数
130     poly A = poly_idev(poly_mul(poly_dev(f), poly_inv(f, deg)));
131     return A.resize(deg), A;
132 }
133
134 poly poly_exp(poly &f, int deg) { //多项式求指数
135     if(deg == 1)
136         return poly(1, 1);
137
138     poly B = poly_exp(f, (deg + 1) >> 1);
139     B.resize(deg);
140     poly lnB = poly_ln(B, deg);
141     for(int i = 0; i < deg; ++ i)
142         lnB[i] = ck(f[i] - lnB[i] + mod);
143
144     int limit = NTT_init(deg << 1); //n -> n^2
145     NTT(B, 1, limit), NTT(lnB, 1, limit);
146     for(int i = 0; i < limit; ++ i)
147         B[i] = 111 * B[i] * (1 + lnB[i]) % mod;
148     NTT(B, 0, limit);
149     B.resize(deg);
150     return B;
151 }
152
153 poly poly_sqrt(poly &f, int deg) { //多项式开方
154     if(deg == 1) return poly(1, 1);
155     poly A(f.begin(), f.begin() + deg);
156     poly B = poly_sqrt(f, (deg + 1) >> 1);
157     poly IB = poly_inv(B, deg);
158     int limit = NTT_init(deg << 1);
159     NTT(A, 1, limit), NTT(IB, 1, limit);
160     for(int i = 0; i < limit; ++ i)
161         A[i] = 111 * A[i] * IB[i] % mod;
162     NTT(A, 0, limit);
163     for(int i = 0; i < deg; ++ i)
164         A[i] = 111 * (A[i] + B[i]) * inv[2] % mod;
165     A.resize(deg);
166     return A;
167 }
168
169 poly poly_pow(poly f, int k) { //多项式快速幂
170     f = poly_ln(f, f.size());
171     for(auto &x : f) x = 111 * x * k % mod;
172     return poly_exp(f, f.size());
173 }
174
175 poly poly_cos(poly f, int deg) { //多项式三角函数 (cos)
176     poly A(f.begin(), f.begin() + deg);
177     poly B(deg), C(deg);
178     for(int i = 0; i < deg; ++ i)

```

```

179     A[i] = 111 * A[i] * img % mod;
180
181     B = poly_exp(A, deg);
182     C = poly_inv(B, deg);
183     int inv2 = qpow(2, mod - 2);
184     for(int i = 0; i < deg; ++ i)
185         A[i] = 111 * (111 * B[i] + C[i]) % mod * inv2 % mod;
186     return A;
187 }
188
189 poly poly_sin(poly f, int deg) { //多项式三角函数 (sin)
190     poly A(f.begin(), f.begin() + deg);
191     poly B(deg), C(deg);
192     for(int i = 0; i < deg; ++ i)
193         A[i] = 111 * A[i] * img % mod;
194
195     B = poly_exp(A, deg);
196     C = poly_inv(B, deg);
197     int inv2i = qpow(img << 1, mod - 2);
198     for(int i = 0; i < deg; ++ i)
199         A[i] = 111 * (111 * B[i] - C[i] + mod) % mod * inv2i % mod;
200     return A;
201 }
202
203 poly poly_arcsin(poly f, int deg) {
204     poly A(f.size(), B(f.size()), C(f.size()));
205     A = poly_dev(f);
206     B = poly_mul(f, f);
207     for(int i = 0; i < deg; ++ i)
208         B[i] = minus(mod, B[i]);
209     B[0] = plus(B[0], 1);
210     C = poly_sqrt(B, deg);
211     C = poly_inv(C, deg);
212     C = poly_mul(A, C);
213     C = poly_iddev(C);
214     return C;
215 }
216
217 poly poly_arctan(poly f, int deg) {
218     poly A(f.size(), B(f.size()), C(f.size()));
219     A = poly_dev(f);
220     B = poly_mul(f, f);
221     B[0] = plus(B[0], 1);
222     C = poly_inv(B, deg);
223     C = poly_mul(A, C);
224     C = poly_iddev(C);
225     return C;
226 }
227 }
228
229 using Poly::poly;
230 using Poly::poly_arcsin;
231 using Poly::poly_arctan;
232
233 int n, m, x, k, type;
234 poly f, g;
235 char s[N];
236
237 int main()
238 {
239     Poly::init(18); //2^21 = 2,097,152, 根据题目数据多项式项数的大小自由调整, 注意大小需要跟deer数组同步(21+1=22)
240
241     read(n), read(type);
242
243     for(int i = 0; i < n; ++ i)
244         read(x), f.push_back(x);
245
246     if(type == 0) g = poly_arcsin(f, n);
247     else g = poly_arctan(f, n);
248
249     for(int i = 0; i < n; ++ i)
250         printf("%d ", g[i]);
251     return 0;
252 }
253 /*

```

```

254 namespace poly_base (变换基础)
255 namespace poly (多项式初等函数)
256 namespace CDQ_NTT (分治多项式技巧)
257 namespace poly_evaluation (多点求值)
258 namespace poly_interpolation (快速插值)
259 namespace linear_recur (常系数线性齐次递推式 - Fiduccia)
260 namespace linear_recur_single (常系数线性齐次递推式 - 单点)
261 namespace miscellaneous (杂项)
262 */
263 // This is a NEW polynomial template for C++11
264 #include <bits/stdc++.h>
265 #define EB emplace_back
266 #define lg2 std::__lg
267
268 typedef unsigned long long u64;
269 const int N = 530000, mod = 998244353, iv2 = (mod + 1) / 2, unity = 31;
270 typedef int vec[N], *pvec;
271 typedef std::pair<int, int> pr;
272 typedef std::vector<int> vector;
273
274 vec inv, fact, finv;
275
276 inline int min(const int x, const int y) {return x < y ? x : y;}
277 inline int max(const int x, const int y) {return x < y ? y : x;}
278 inline int & reduce(int &x) {return x += x >> 31 & mod;}
279 inline int & neg(int &x) {return x = (!x - 1) & (mod - x);}
280 u64 PowerMod(u64 a, int n, u64 c = 1) {for (; n >= 1, a = a * a % mod; if (n & 1) c = c * a % mod; return c;}
281
282 namespace poly_base {
283     int l, n; u64 iv; vec w2;
284
285     void init(int n = N, bool dont_calc_factorials = true) {
286         int i, t;
287         for (inv[1] = 1, i = 2; i < n; ++i) inv[i] = u64(mod - mod / i) * inv[mod % i] % mod;
288         if (!dont_calc_factorials) for (*finv = *fact = i = 1; i < n; ++i) fact[i] = (u64)fact[i - 1] * i %
mod, finv[i] = (u64)finv[i - 1] * inv[i] % mod;
289         t = min(n > 1 ? lg2(n - 1) : 0, 21),
290         *w2 = 1, w2[1 << t] = PowerMod(unity, 1 << (21 - t));
291         for (i = t; i; --i) w2[1 << (i - 1)] = (u64)w2[1 << i] * w2[1 << i] % mod;
292         for (i = 1; i < n; ++i) w2[i] = (u64)w2[i & (i - 1)] * w2[i & -i] % mod;
293     }
294
295     inline void NTT_init(int len) {n = 1 << (l = len), iv = mod - (mod - 1) / n;}
296
297     void DIF(int *a) {
298         int i, *j, *k, len = n >> 1, R, *o;
299         for (i = 0; i < l; ++i, len >= 1)
300             for (j = a, o = w2; j != a + n; j += len << 1, ++o)
301                 for (k = j; k != j + len; ++k)
302                     R = (u64)*o * k[len] % mod, reduce(k[len] = *k - R), reduce(*k += R - mod);
303     }
304
305     void DIT(int *a) {
306         int i, *j, *k, len = 1, R, *o;
307         for (i = 0; i < l; ++i, len <= 1)
308             for (j = a, o = w2; j != a + n; j += len << 1, ++o)
309                 for (k = j; k != j + len; ++k)
310                     reduce(R = *k + k[len] - mod), k[len] = u64(*k - k[len] + mod) * *o % mod, *k = R;
311     }
312
313     inline void DNTT(int *a) {DIF(a);}
314     inline void IDNTT(int *a) {
315         DIT(a), std::reverse(a + 1, a + n);
316         for (int i = 0; i < n; ++i) a[i] = a[i] * iv % mod;
317     }
318
319     inline void DIF(int *a, int *b) {memcpy(b, a, n << 2), DIF(b);}
320     inline void DIT(int *a, int *b) {memcpy(b, a, n << 2), DIT(b);}
321     inline void DNTT(int *a, int *b) {memcpy(b, a, n << 2), DNTT(b);}
322     inline void IDNTT(int *a, int *b) {memcpy(b, a, n << 2), IDNTT(b);}
323 }
324
325 namespace poly {
326     using namespace poly_base;

```

```

327
328     vec B1, B2, B3, B4, B5, B6;
329
330     // Multiplication (use one buffer, 3-dft of length 2n)
331     void mul(int deg, pvec a, pvec b, pvec c) {
332         if (!deg) { *c = (u64)*a * *b % mod; return; }
333         NTT_init(lg2(deg) + 1), DNTT(a, c), DNTT(b, B1);
334         for (int i = 0; i < n; ++i) c[i] = (u64)c[i] * B1[i] % mod;
335         IDNTT(c);
336     }
337
338     // Inversion (use three buffers, 5-dft)
339     void inv(int deg, pvec a, pvec b) {
340         int i, len; assert(*a);
341         if (*b = PowerMod(*a, mod - 2), deg <= 1) return;
342         memset(b + 1, 0, i = 8 << lg2(deg - 1)), memset(B1, 0, i), *B1 = *a;
343
344         for (len = 0; 1 << len < deg; ++len) {
345             NTT_init(len + 1);
346
347             memcpy(B1 + (n >> 1), a + (n >> 1), n << 1), DIF(b, B2), DIF(B1, B3);
348             for (i = 0; i < n; ++i) B3[i] = (u64)B3[i] * B2[i] % mod; DIT(B3);
349             for (i = n >> 1; i < n; ++i) B3[i] = B3[n - i] * iv % mod;
350
351             memset(B3, 0, n << 1), DIF(B3);
352             for (i = 0; i < n; ++i) B3[i] = (u64)B3[i] * B2[i] % mod; DIT(B3);
353             for (i = n >> 1; i < n; ++i) b[i] = B3[n - i] * (mod - iv) % mod;
354         }
355     }
356
357     // Division and Modulo Operation (use five buffers)
358     void div_mod(int A, int B, pvec a, pvec b, pvec q, pvec r) {
359         if (A < B) { memcpy(r, a, (A + 1) << 2), memset(r + (A + 1), 0, (B - A) << 2); return; }
360         int Q = A - B, i, l_ = Q ? lg2(Q) + 1 : 0; NTT_init(l_);
361         for (i = 0; i <= Q && i <= B; ++i) B4[i] = b[B - i];
362         memset(B4 + i, 0, (n - i) << 2), inv(i = Q + 1, B4, B5);
363
364         std::reverse_copy(a + B, a + (A + 1), B4), NTT_init(++l_);
365         memset(B4 + i, 0, (n - i) << 2), memset(B5 + i, 0, (n - i) << 2),
366         mul(2 * Q, B4, B5, q), std::reverse(q, q + (Q + 1)),
367         memset(q + i, 0, (n - i) << 2);
368
369         if (!B) return;
370         NTT_init(lg2(2 * B - 1) + 1);
371         for (i = 0; i <= Q && i < B; ++i) B2[i] = b[i], B3[i] = q[i];
372         memset(B2 + i, 0, (n - i) << 2), memset(B3 + i, 0, (n - i) << 2),
373         mul(2 * (B - 1), B2, B3, r), memset(r + i, 0, (n - i) << 2);
374         for (i = 0; i < B; ++i) reduce(r[i] = a[i] - r[i]);
375     }
376
377     // Multiplication with std::vector (use two buffers, 3-dft)
378     void mul(vector &a, vector &b, vector &ret) {
379         int A = a.size() - 1, B = b.size() - 1;
380         if (!(A || B)) { ret.EB((u64)a[0] * b[0] % mod); return; }
381         NTT_init(lg2(A + B) + 1),
382         memcpy(B1, a.data(), (A + 1) << 2), memset(B1 + (A + 1), 0, (n - A - 1) << 2),
383         memcpy(B2, b.data(), (B + 1) << 2), memset(B2 + (B + 1), 0, (n - B - 1) << 2),
384         DNTT(B1), DNTT(B2);
385         for (int i = 0; i < n; ++i) B1[i] = (u64)B1[i] * B2[i] % mod;
386         IDNTT(B1), ret.assign(B1, B1 + (A + B + 1));
387     }
388
389     // Differential
390     void diff(int deg, pvec a, pvec b) { for (int i = 1; i <= deg; ++i) b[i - 1] = (u64)a[i] * i % mod; }
391
392     // Integral
393     void intg(int deg, pvec a, pvec b, int constant = 0) { for (int i = deg; i; --i) b[i] = (u64)a[i - 1] *
394     ::inv[i] % mod; *b = constant; }
395
396     // f'[x] / f[x] (use four buffers, 6.5-dft)
397     void dif_quo(int deg, pvec a, pvec b) {
398         assert(*a);
399         if (deg <= 1) { *b = PowerMod(*a, mod - 2, a[1]); return; }
400
401         int i, len = lg2(deg - 1);

```

```

401     inv((deg + 1) / 2, a, B4), NTT_init(len + 1),
402     memset(B4 + (n >> 1), 0, n << 1), DIF(B4, B2),
403
404     diff(deg, a, B1), memcpy(B3, B1, n << 1),
405     memset(B3 + (n >> 1), 0, n << 1), DIF(B3);
406
407     for (i = 0; i < n; ++i) B3[i] = (u64)B3[i] * B2[i] % mod;
408     DIT(B3, b), *b = *b * iv % mod;
409     for (i = 1; i < n >> 1; ++i) b[i] = b[n - i] * iv % mod;
410     memset(b + (n >> 1), 0, n << 1);
411
412     DIF(b, B4), DIF(a, B3);
413     for (i = 0; i < n; ++i) B3[i] = (u64)B3[i] * B4[i] % mod; DIT(B3);
414     for (i = n >> 1; i < n; ++i) B3[i] = (B3[n - i] * iv + mod - B1[i]) % mod;
415
416     memset(B3, 0, n << 1), DIF(B3);
417     for (i = 0; i < n; ++i) B3[i] = (u64)B3[i] * B2[i] % mod; DIT(B3);
418     for (i = n >> 1; i < n; ++i) b[i] = B3[n - i] * (mod - iv) % mod;
419 }
420
421 // Logarithm (use DifQuo)
422 inline void ln(int deg, pvec a, pvec b) {assert(*a == 1), --deg ? (dif_quo(deg, a, b), intg(deg, b, b)) :
void(*b = 0);}
423
424 // Exponentiation (use six buffers, 12-dft)
425 // WARNING : this implementation of exponentiation is SLOWER than the CDQ NTT ver.
426 void exp(int deg, pvec a, pvec b) {
427     int i, len; pvec c = B6; assert(*a);
428     if (*b = 1, deg <= 1) return;
429     if (b[1] = a[1], deg == 2) return;
430
431     memset(b + 2, 0, i = 8 << lg2(deg - 1)), memset(c, 0, i), memset(B1, 0, i),
432     *c = 1, neg(c[1] = b[1]);
433
434     for (len = 1; 1 << len < deg; ++len) {
435         NTT_init(len + 1);
436
437         DIF(c, B2), DIF(b, B3);
438         for (i = 0; i < n; ++i) B4[i] = (u64)B3[i] * B2[i] % mod; DIT(B4);
439         for (i = n >> 1; i < n; ++i) B4[i] = B4[n - i] * iv % mod;
440
441         memset(B4, 0, n << 1), DIF(B4);
442         for (i = 0; i < n; ++i) B4[i] = (u64)B4[i] * B2[i] % mod; DIT(B4);
443         for (i = n >> 1; i < n; ++i) B4[i] = B4[n - i] * (mod - iv) % mod;
444
445         memcpy(B4, c, n << 1), DIF(B4);
446         diff(n >> 1, b, B1), DIF(B1, B5);
447         for (i = 0; i < n; ++i) B4[i] = (u64)B4[i] * B5[i] % mod; DIT(B4);
448         for (i = n >> 1; i < n; ++i) reduce(B5[i] = (a[i] + B4[n - i + 1] * (mod - iv) % mod * ::inv[i]) %
mod);
449
450         memset(B5, 0, n << 1), DIF(B5);
451         for (i = 0; i < n; ++i) B5[i] = (u64)B5[i] * B3[i] % mod; DIT(B5);
452         for (i = n >> 1; i < n; ++i) b[i] = B5[n - i] * iv % mod;
453
454         if (2 << len >= deg) return;
455
456         DIF(b, B3);
457         for (i = 0; i < n; ++i) B3[i] = (u64)B3[i] * B2[i] % mod; DIT(B3);
458         for (i = n >> 1; i < n; ++i) B3[i] = B3[n - i] * iv % mod;
459
460         memset(B3, 0, n << 1), DIF(B3);
461         for (i = 0; i < n; ++i) B3[i] = (u64)B3[i] * B2[i] % mod; DIT(B3);
462         for (i = n >> 1; i < n; ++i) c[i] = B3[n - i] * (mod - iv) % mod;
463     }
464 }
465 }
466
467 namespace CDQ NTT {
468     using namespace poly_base;
469
470     int lim;
471     vec f, g, C1;
472     int fn[N * 2], gn[N * 2];
473

```

```

474     inline void register_g(pvec g) {for (int i = 1; 1 << (i - 1) <= lim; ++i) NTT_init(i), DIF(g, gn + (1 <<
i));}
475
476     // Standard CDQ-NTT Algorithm, type `UK`
477     void solve(int L, int w) {
478         int i, R = L + (1 << w), M;
479         if (!w) {
480             // something depend on problem
481             return;
482         }
483         solve(L, w - 1);
484         if ((M = (1 << (w - 1)) + L) > lim) return;
485         NTT_init(w);
486         pvec ga = gn + (1 << w);
487         memcpy(C1, f + L, 2 << w), memset(C1 + (1 << (w - 1)), 0, 2 << w),
488         DIF(C1);
489         for (i = 0; i < n; ++i) C1[i] = (u64)C1[i] * ga[i] % mod;
490         DIT(C1);
491         for (i = M; i < R; ++i) f[i] = (f[i] + C1[n - (i - L)] * iv) % mod;
492         solve(M, w - 1);
493     }
494
495     void solve(int L, int w) {
496         int i, R = L + (1 << w), M;
497         if (!w) {
498             // something depend on problem
499             return;
500         }
501         solve(L, w - 1);
502         if ((M = (1 << (w - 1)) + L) > lim) return;
503         NTT_init(w);
504         if (L) {
505             pvec fa = fn + (1 << w), ga = gn + (1 << w);
506             memcpy(C1, f + L, 2 << w), memset(C1 + (1 << (w - 1)), 0, 2 << w),
507             memcpy(C2, g + L, 2 << w), memset(C2 + (1 << (w - 1)), 0, 2 << w),
508             DIF(C1), DIF(C2);
509             for (i = 0; i < n; ++i) C1[i] = ((u64)C1[i] * ga[i] + (u64)C2[i] * fa[i]) % mod;
510             DIT(C1);
511             for (i = M; i < R; ++i) f[i] = (f[i] + C1[n - (i - L)] * iv) % mod;
512         } else {
513             memcpy(C1, f, 2 << w), memset(C1 + M, 0, 2 << w),
514             memcpy(C2, g, 2 << w), memset(C2 + M, 0, 2 << w),
515             DIF(C1), DIF(C2),
516             memcpy(fn + (1 << (w - 1)), C1, 2 << w),
517             memcpy(gn + (1 << (w - 1)), C2, 2 << w);
518             for (i = 0; i < n; ++i) C1[i] = (u64)C1[i] * C2[i] % mod;
519             DIT(C1);
520             for (i = M; i < R; ++i) f[i] = (f[i] + C1[n - i] * iv) % mod;
521         }
522         solve(M, w - 1);
523     }
524 }
525
526 namespace poly_evaluation {
527     using namespace poly_base;
528
529     int cnt = 0, lc[N], rc[N];
530     vec Prd_, E1, E2, E3;
531     vector g[N], tmp_;
532
533     int solve(int L, int R) {
534         if (L + 1 == R) return L;
535         int M = (L + R) / 2, id = cnt++, lp = solve(L, M), rp = solve(M, R);
536         return poly::mul(g[lp], g[rp], g[id]), lc[id] = lp, rc[id] = rp, id;
537     }
538
539     void recursion(int id, int L, int R, const vector &poly) {
540         if (L + 1 == R) return tmp_.EB(poly.back());
541         int i, n = poly.size() - 1, M = (L + R) / 2, lp = lc[id], rp = rc[id],
542         dl = min(n, g[lp].size() - 1), dr = min(n, g[rp].size() - 1);
543         if (L + 2 == R) return
544             tmp_.EB((poly[n] + (u64)poly[n - 1] * g[rp].back()) % mod),
545             tmp_.EB((poly[n] + (u64)poly[n - 1] * g[lp].back()) % mod);
546
547         vector ly, ry; ly.reserve(dl + 1), ry.reserve(dr + 1);

```

```

548     NTT_init(lg2(d1 + dr) + 1);
549     memcpy(E1, poly.data(), (n + 1) << 2), DIF(E1, E2), memset(E1, 0, (n + 1) << 2);
550
551     memcpy(E1, g[rp].data(), (dr + 1) << 2), DIF(E1, E3), memset(E1, 0, (dr + 1) << 2);
552     for (i = 0; i < poly::n; ++i) E3[i] = (u64)E3[i] * E2[i] % mod;
553     DIT(E3), std::reverse(E3 + 1, E3 + poly::n);
554     for (i = n - d1; i <= n; ++i) ly.EB(E3[i] * iv % mod);
555
556     memcpy(E1, g[lp].data(), (d1 + 1) << 2), DIF(E1, E3), memset(E1, 0, (d1 + 1) << 2);
557     for (i = 0; i < poly::n; ++i) E3[i] = (u64)E3[i] * E2[i] % mod;
558     DIT(E3), std::reverse(E3 + 1, E3 + poly::n);
559     for (i = n - dr; i <= n; ++i) ry.EB(E3[i] * iv % mod);
560
561     recursion(lp, L, M, ly), recursion(rp, M, R, ry);
562 }
563
564 vector emain(int n, pvec f, const vector &pts) {
565     int i, id, m = pts.size(), q;
566     if (!m) return vector();
567     if (!n) return vector(m, *f);
568     for (i = 0; i < m; ++i) g[i].clear(), g[i].EB(1), g[i].EB(neg(q = pts[i]));
569
570     id = solve(0, cnt = m), memcpy(Prd_, g[id].data(), (m + 1) << 2);
571     poly::inv(n + 1, Prd_, E2), memset(Prd_, 0, (m + 1) << 2);
572     if (n > 0) memset(E2 + (n + 1), 0, (poly::n - n - 1) << 2);
573
574     std::reverse_copy(f, f + (n + 1), E1), poly::mul(2 * n, E1, E2, E3),
575     memset(E1, 0, (n + 1) << 2), memset(E2, 0, (n + 1) << 2);
576
577     return tmp_.clear(), tmp_.reserve(m), recursion(id, 0, m, vector(E3 + max(n - m + 1, 0), E3 + (n +
1))), tmp_;
578 }
579 }
580
581 namespace poly_interpolation {
582     using namespace poly_evaluation;
583
584     vec I1, I2, I3, I4, I5;
585     pvec iresult;
586
587     void irecursion(int id, int L, int R) {
588         if (L + 1 == R) return;
589         int i, M = (L + R) / 2, lp = lc[id], rp = rc[id];
590         irecursion(lp, L, M), irecursion(rp, M, R);
591         NTT_init(lg2(R - L - 1) + 1);
592         memcpy(I1, iresult + L, (M - L) << 2), memset(I1 + (M - L), 0, (poly::n - (M - L)) << 2);
593         memcpy(I2, iresult + M, (R - M) << 2), memset(I2 + (R - M), 0, (poly::n - (R - M)) << 2);
594         memcpy(I3, g[lp].data(), (M - L + 1) << 2), memset(I3 + (M - L + 1), 0, (poly::n - (M - L + 1)) << 2);
595         memcpy(I4, g[rp].data(), (R - M + 1) << 2), memset(I4 + (R - M + 1), 0, (poly::n - (R - M + 1)) << 2);
596         DIF(I1), DIF(I2), DIF(I3), DIF(I4);
597         for (i = 0; i < n; ++i) I1[i] = ((u64)I1[i] * I4[i] + (u64)I2[i] * I3[i]) % mod;
598         DIT(I1), std::reverse(I1 + 1, I1 + n);
599         for (i = 0; i < R - L; ++i) iresult[L + i] = I1[i] * iv % mod;
600     }
601
602     void imain(int n, pr *pts, pvec ret) {
603         int i, id, q;
604         assert(n > 0);
605         for (i = 0; i < n; ++i) g[i].clear(), g[i].EB(1), g[i].EB(neg(q = pts[i].first));
606
607         id = solve(0, cnt = n), memcpy(Prd_, g[id].data(), (n + 1) << 2);
608         poly::inv(n, Prd_, E2), memset(Prd_, 0, n << 2);
609         if (n > 1) memset(E2 + n, 0, (poly::n - n) << 2);
610
611         for (i = 0; i < n; ++i) E1[i] = u64(n - i) * g[id][i] % mod;
612         poly::mul(2 * (n - 1), E1, E2, E3),
613         memset(E1, 0, n << 2), memset(E2, 0, n << 2);
614
615         tmp_.clear(), tmp_.reserve(n), recursion(id, 0, n, vector(E3, E3 + n));
616         for (i = 0; i < n; ++i) ret[i] = PowerMod(tmp_[i], mod - 2, pts[i].second);
617         iresult = ret, irecursion(id, 0, n), std::reverse(ret, ret + n);
618     }
619 }
620
621 namespace linear_recur {

```

```

622     using namespace poly_base;
623
624     int ld;
625     vec f, g, L1, L2, L3;
626
627     void __builtin_divmod(int d) {
628         int i; NTT_init(ld);
629         std::reverse_copy(f + d, f + 2 * d, L3);
630         memset(L3 + d, 0, (n - d) << 2);
631         DNTT(L3);
632         for (i = 0; i < n; ++i) L3[i] = (u64)L3[i] * L2[i] % mod;
633         IDNTT(L3);
634         std::reverse(L3, L3 + d), memset(L3 + d, 0, (n - d) << 2);
635         DNTT(L3);
636         for (i = 0; i < n; ++i) L3[i] = (u64)L3[i] * L1[i] % mod;
637         IDNTT(L3);
638         for (i = 0; i < d; ++i) reduce(f[i] -= L3[i]);
639         memset(f + d, 0, d << 2);
640     }
641
642     void solve(int Q, int d) {
643         int i, df = 1, z = lg2(Q); bool alive = false;
644         assert(d > 0), ld = lg2(2 * d - 1) + 1;
645         if (d == 1) alive = true, *f = 1;
646
647         NTT_init(ld),
648         std::reverse_copy(g + 1, g + (d + 1), L1),
649         memset(L1 + d, 0, (n - d) << 2),
650         poly::inv(d, L1, L2);
651
652         NTT_init(ld),
653         memset(L2 + d, 0, (n - d) << 2),
654         memcpy(L1, g, d << 2),
655         memset(L1 + d, 0, (n - d) << 2),
656         DIF(L1), DIF(L2);
657
658         for (; --z >= 0; ) {
659             df = df << 1 | (Q >> z & 1);
660             if (df < d) continue;
661             if (!alive) alive = true, f[df >> 1] = 1;
662             NTT_init(ld), DNTT(f);
663             for (i = 0; i < n; ++i) f[i] = (u64)f[i] * f[i] % mod;
664             IDNTT(f);
665             if (df & 1) std::copy_backward(f, f + (2 * d - 1), f + 2 * d), *f = 0;
666             __builtin_divmod(d);
667         }
668     }
669 }
670
671 namespace linear_recur_single {
672     using namespace poly;
673
674     int ld;
675     vec f, g, L1, L2, L3, L4;
676
677     int get(int Q, int d) {
678         if (Q < d) return f[Q];
679
680         int i, j, ret = 0, halves = 0;
681         assert(d > 0), ld = lg2(2 * d - 1) + 1,
682
683         mul(2 * d - 1, f, g, L1), memset(L1 + d, 0, (n - d) << 2),
684         memcpy(L2, g, (d + 1) << 2), memset(L2 + (d + 1), 0, (n - d - 1) << 2);
685
686         for (; Q >= d; Q >= 1) {
687             NTT_init(ld), DIF(L1, L4), DIF(L2, L3),
688             NTT_init(ld - 1), ++halves;
689             if (Q & 1) {
690                 for (j = i = 0; i < n; ++i, j += 2)
691                     L4[i] = ((u64)L4[j] * L3[j + 1] + u64(mod - L3[j]) * L4[j + 1]) % mod;
692                 for (i = 0; i < n; ++i) L4[i] = (u64)L4[i] * w2[i] % mod;
693                 IDNTT(L4), L4[n] = *L4, memcpy(L1, L4 + 1, d << 2);
694             } else {
695                 for (j = i = 0; i < n; ++i, j += 2)
696                     L4[i] = ((u64)L4[j] * L3[j + 1] + (u64)L3[j] * L4[j + 1]) % mod;

```

```

697         IDNTT(L4, L1);
698     }
699     for (j = i = 0; i < n; ++i, j += 2) L3[i] = (u64)L3[j] * L3[j + 1] % mod;
700     IDNTT(L3, L2);
701     if (*L2 != 1) L2[d] = (*L2 * L2 : mod) - 1, *L2 = 1;
702 }
703
704 poly::inv(Q + 1, L2, L3);
705 for (i = 0; i <= Q; ++i) ret = (ret + (u64)L1[i] * L3[Q - i]) % mod;
706 return PowerMod(iv2, halves, ret);
707 }
708 }
709
710 namespace miscellaneous {
711     using namespace poly_base;
712
713     vec M1, M2, M3, M4;
714     vec Ma[200];
715
716     // calculate b(x) = a(x + c), poly_base::init need 'false'
717     void translate(int deg, pvec a, int c, pvec b) {
718         if (!deg) { *b = *a; return; }
719         int i; u64 pc = 1; NTT_init(lg2(deg) + 2);
720         for (i = 0; i <= deg; ++i) M1[i] = (u64)a[i] * fact[i] % mod;
721         memset(M1 + i, 0, (n - i) << 2);
722         for (i = deg; i >= 0; --i, pc = pc * c % mod) M2[i] = finv[deg - i] * pc % mod;
723         DIF(M1), DIF(M2);
724         for (i = 0; i < n; ++i) M1[i] = (u64)M1[i] * M2[i] % mod;
725         DIT(M1);
726         for (i = 0; i <= deg; ++i) b[i] = M1[n - deg - i] * iv % mod * finv[i] % mod;
727     }
728
729     // compute a(1), a(z), a(z^2), ..., a(z^(len-1)) to b
730     void czt(int deg, int len, int z, pvec a, pvec b) {
731         int i; u64 iz = PowerMod(z, mod - 2);
732         if (!len) return;
733         if (!deg || !z) return std::fill(b, b + len, *a);
734         NTT_init(lg2(deg + len - 1) + 1);
735         M1[1] = *M1 = M3[1] = *M3 = 1;
736         for (i = 2; i < deg + len; ++i) M1[i] = M1[i - 1] * (u64)z % mod, M3[i] = M3[i - 1] * iz % mod;
737         for (i = 2; i < deg + len; ++i) M1[i] = (u64)M1[i] * M1[i - 1] % mod, M3[i] = (u64)M3[i] * M3[i - 1] %
mod;
738         memset(M1 + i, 0, (n - i) << 2);
739         for (i = 0; i <= deg; ++i) M2[deg - i] = (u64)M3[i] * a[i] % mod;
740         memset(M2 + i, 0, (n - i) << 2);
741         DIF(M1), DIF(M2);
742         for (i = 0; i < n; ++i) M1[i] = (u64)M1[i] * M2[i] % mod;
743         DIT(M1);
744         for (i = 0; i < len; ++i) b[i] = M1[n - deg - i] * iv % mod * M3[i] % mod;
745     }
746
747     inline int __builtin_inner(int len, pvec a, pvec b) {
748         int i = 0; u64 ans = 0;
749         #define term(k) (u64)a[i + k] * b[i + k]
750         for (; i + 15 < len; i += 16) {
751             ans = (ans + term(0) + term(1) + term(2) + term(3)
752                 + term(4) + term(5) + term(6) + term(7)
753                 + term(8) + term(9) + term(10) + term(11)
754                 + term(12) + term(13) + term(14) + term(15)
755                 ) % mod;
756         }
757         #undef term
758         for (; i < len; ++i) ans += (u64)a[i] * b[i];
759         return ans % mod;
760     }
761
762     // compositional inverse
763     void cinv(int deg, pvec a, pvec b) {
764         assert(!*a), *b = 0;
765         if (--deg <= 0) return;
766         int i, j, k, d, B, *c;
767
768         NTT_init(lg2(2 * deg - 1)),
769         memcpy(M1, a + 1, deg << 2), memset(M1 + deg, 0, (n - deg) << 2),
770         poly::inv(deg, M1, M2),

```

```

771     NTT_init(lg2(2 * deg - 1) + 1),
772     memset(M2 + deg, 0, (n - deg) << 2),
773
774
775     DIF(M2, M1), B = sqrt(deg),
776     memset(*Ma, 0, n << 2), **Ma = 1;
777     for (j = 1; j <= B; ++j) {
778         DIF(Ma[j - 1], M3), c = Ma[j];
779         for (i = 0; i < n; ++i) M3[i] = (u64)M3[i] * M1[i] % mod;
780         DIT(M3), *c = *M3 * iv % mod;
781         for (i = 1; i < deg; ++i) c[i] = M3[n - i] * iv % mod;
782         memset(c + i, 0, (n - i) << 2);
783     }
784     DIF(Ma[B], M1), memset(M3, 0, n << 2), *M3 = M4[deg - 1] = 1;
785
786     for (d = 0; ; ) {
787         for (k = 0; k < B && d < deg; ++k, ++d)
788             b[d + 1] = (u64)_builtin_inner(d + 1, Ma[k + 1], M4 + (deg - d - 1)) * inv[d + 1] % mod;
789         if (d >= deg) break;
790         DIF(M3);
791         for (i = 0; i < n; ++i) M3[i] = (u64)M3[i] * M1[i] % mod;
792         DIT(M3), *M3 = *M3 * iv % mod;
793         for (i = 1; i < deg; ++i) M3[i] = M3[n - i] * iv % mod;
794         memset(M3 + i, 0, (n - i) << 2),
795         std::reverse_copy(M3, M3 + deg, M4);
796     }
797 }
798 }
799
800

```

组合数学

排列组合

性质:

1. 对称性: $\binom{n}{m} = \binom{n}{n-m} \Leftrightarrow C_n^m = C_n^{n-m}$

2. 递推式:

1. 定义递推式: $\binom{n}{m} = \frac{n}{m} \binom{n-1}{m-1} \Leftrightarrow C_n^m = \frac{n}{m} C_{n-1}^{m-1}$

2. 杨辉三角递推: $\binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1} \Leftrightarrow C_n^m = C_{n-1}^m + C_{n-1}^{m-1}$

$$\sum_{i=1}^k \binom{n+i}{i} = \sum_{i=1}^k \binom{n+i}{n-i}$$

$$= \left[\binom{n+1}{n+1} + \binom{n+1}{n} \right] - \binom{n+1}{n+1} + \sum_{i=2}^k \binom{n+i}{n-i}$$

3. 递推消求和符号:

$$= \left[\binom{n+2}{n+1} + \binom{n+2}{n} \right] - \binom{n+1}{n+1} + \sum_{i=3}^k \binom{n+i}{n-i}$$

$\vdots$

$$= \binom{n+k+1}{n+1} - \binom{n+1}{n+1} = \binom{n+k+1}{n+1} - 1$$

3. 二项式定理:

1. $\forall a, b: \sum_{i=0}^n a^i b^{n-i} \binom{n}{i} = (a+b)^n = \sum_{i=0}^n a^i b^{n-i} C_n^i$

2. $a = b = 1: \binom{n}{0} + \binom{n}{1} + \cdots + \binom{n}{n} = \sum_{i=0}^n \binom{n}{i} = 2^n = \sum_{i=0}^n C_n^i = C_n^0 + C_n^1 + \cdots + C_n^n$

3. $a = -b = 1: \sum_{i=0}^n (-1)^i \binom{n}{i} = [n=0] = \sum_{i=0}^n (-1)^i C_n^i$

4. 范德蒙德卷积: $\sum_{i=0}^k \binom{n}{i} \binom{m}{k-i} = \sum_{i=0}^k \binom{n}{i} \binom{m}{i} = \binom{n+m}{k} = \sum_{i=0}^k C_n^i C_m^i = \sum_{i=0}^k C_n^i C_m^{k-i}$

1. 组合意义: 在 $n + m$ 中选出 k 个数 $\Leftrightarrow$ 在 n 中选出 i 个数, 在 m 中选出 $k - i$ 个数

$$\begin{aligned}
& \sum_{k=0}^{n+m} \binom{n+m}{k} x^k \\
&= (x+1)^{n+m} \\
2. \text{ 二项式定理: } &= (x+1)^n (x+1)^m \\
&= \sum_{i=0}^n \binom{n}{i} x^i \sum_{s=0}^m \binom{m}{s} x^s, \text{ 令 } k=i+s \\
&= \sum_{k=0}^{n+m} \sum_{i=0}^n \binom{n}{i} \binom{m}{k-i} x^k
\end{aligned}$$

◦ 推论:

$$\begin{aligned}
1. \sum_{i=-r}^s \binom{n}{r+i} \binom{m}{s-i} &= \binom{n+m}{r+s} \\
2. \sum_{i=1}^n \binom{n}{i} \binom{n}{i-1} &= \binom{2n}{n-1} \\
3. \sum_{i=0}^n \binom{n}{i}^2 &= \sum_{i=0}^n \binom{n}{i} \binom{n}{n-i} = \binom{2n}{n} \\
4. \text{ 拆组合数: } \sum_{i=0}^m \binom{n}{i} \binom{m}{i} &= \sum_{i=0}^m \binom{n}{i} \binom{m}{m-i} = \binom{n+m}{m}, n \geq m
\end{aligned}$$

+ $\binom{n+m}{m}$ 是网格图路径从 $(0,0)$ 走到 (n,m) 的方案数,向右走了 m 步,向下走了 n 步, 那么 n 步中有 i 步向右,那 m 步中就有 $m-i$ 步向右

5. 多项式函数:

$$\begin{aligned}
1. \sum_{i=0}^n i \binom{n}{i} &= n2^{n-1} \\
2. \sum_{i=0}^n i^2 \binom{n}{i} &= n(n+1)2^{n-2}
\end{aligned}$$

6. 子集数: $\sum_{l=0}^n \binom{l}{k} = \binom{n+1}{k+1}$, 考虑 $S = \{a_1, a_2, \dots, a_{n+1}\}$ 的 $k+1$ 子集数即可证明

7. 定义法: $\binom{n}{r} \binom{r}{k} = \binom{n}{k} \binom{n-k}{r-k}$

8. 斐波那契: $\sum_{i=0}^n \binom{n-i}{i} = F_{n+1}$

插板法:

1. 线性不定方程: 求解 $\sum_{i=1}^k x_i = n$ 的方案数问题

1. 正整数解方案数: $\binom{n-1}{k-1} \Leftrightarrow$ 在 $n-1$ 个空隙中插入 $k-1$ 个板分隔, 将 n 个完全相同的元素分成 k 组, 每组至少一个元素

2. 非负整数解的数目: $\binom{n+k-1}{k-1} = \binom{n+k-1}{n} \Leftrightarrow$ 向外面借 k 个元素, 将 n 个完全相同的元素分成 k 组, 每组可为空

3. 不同下界整数和的数目: $\binom{n-\sum a_i+k-1}{n-\sum a_i} \Leftrightarrow$ 使第 i 组至少分到 a_i 个元素, 令

$$x'_i = x_i - a_i \geq 0 \Rightarrow x'_1 + x'_2 + \dots + x'_k = n - \sum a_i$$

4. 不同上界整数和的数目: 使第 i 组至多分到 a_i 个元素, 考虑容斥, 设满足 $x_i \leq a_i$ 的集合为 S_i , $\overline{S}_i$ 即满足 $x_i \geq a_i + 1$ 的集合, 全集 U 为情况 (ii)

$$\blacksquare \left| \bigcap_{i=1}^k S_i \right| = |U| - \left| \bigcup_{i=1}^k \overline{S}_i \right| = \sum_{p=0}^k (-1)^p \sum_A \binom{k+n-1-\sum_{i \in A} a_{A_i}-p}{k-1} \quad (A \text{ 表示被选的子集})$$

2. 不相邻的排列: $\binom{n-k+1}{k} \Leftrightarrow$ 选出 k 个元素当作隔板, 插入 $n-k+1$ 个空隙中, $1 \sim n$ 这 n 个数中选 k 个, k 个数中俩个数都不相邻

3. 多重集的排列数|多重组合数: 多重集 $S = \{n_1 \cdot a_1, n_2 \cdot a_2, \dots, n_k \cdot a_k\}$ 的全排列方案数为 $\binom{n}{n_1, n_2, \dots, n_k} = \frac{n!}{\prod_{i=1}^k n_i!}$, $\binom{n}{m} = \binom{n}{m, n-m}$

1. 多重集的组合数1: 从多重集 S 中选出 $r (r < n_i)$ 个元素的方案数 $\Leftrightarrow$ 将 r 个元素分成 k 组 $\Leftrightarrow x_1 + x_2 + \dots + x_k = r$ 的非负整数解的方案为 $\binom{r+k-1}{k-1}$

2. 多重集的组合数2: 从多重集 S 中选出 r 个元素的方案数 $\Leftrightarrow$ 将 r 个元素分成 k 组第 i 组不超过 n_i 即情况 1.(iv)

4. 圆排列: $Q_n^r = \frac{A_n^r}{r} = \frac{n!}{r \times (n-r)!}$

实现:

1. 定义&乘法逆元&取模: 由 $C_n^m = \binom{n}{m} = \frac{n!}{m!(n-m)!} = \frac{\prod_{i=0}^{m-1} (n-i)}{m!}$, $A_n^m = \frac{n!}{(n-m)!}$

当 p 为素数时,可用欧拉定理与费马小定理(快速幂求法)

多次询问或 p 较小时可以打逆元表和阶乘的表来多次询问, 分别记为 *inv*, *fact*

非打表计算, 单次时间复杂度 $O(\max\{m, \log p\})$, 若 m 很小, n 很大, 可单独对 m 打表, 避免多次计算, 复杂度不变

```

1 auto C = [](int n, int m, int a = 1, int b = 1) -> int {
2     if (n < m || n < 0 || m < 0) return 0;
3     for (int i = 1; i <= m; i++) {
4         (a *= n - i + 1) %= mod, (b *= i) %= mod;
5     }
6     return a * binpow(b) % mod;
7 };

```

打表计算, 总时间复杂度 $O(n \log p)$

2. **递推&杨辉三角&取模**: 由 $\binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1}$, 可用滚动数组进行优化空间, 多次查询时间复杂度 $O(n^2)$

```

1 for (int i = 0; i <= n; i++) {
2     for (int j = 0; j <= i; j++) {
3         if (!j) {
4             C[i][j] = 1;
5         } else {
6             (C[i][j] = C[i-1][j] + C[i-1][j-1]) %= mod;
7         }
8     }
9 }
10 // 用滚动数组
11 c[0] = 1;
12 for (ll i = 1; i <= n; i++) {
13     for (ll j = i; j; j--) {
14         c[j] = (c[j] + c[j-1]) % mod;
15     }
16 }

```

Lucas定理: 当模数不大且为质数时, 一般要求 p 在 10^5 左右

$$\Rightarrow \binom{n}{m} \bmod p = \binom{\lfloor \frac{n}{p} \rfloor}{\lfloor \frac{m}{p} \rfloor} \cdot \binom{n \bmod p}{m \bmod p} \bmod p$$

且 $\binom{\lfloor \frac{n}{p} \rfloor}{\lfloor \frac{m}{p} \rfloor}$ 可继续用 *Lucas* 求解, 边界条件 $m = 0$ 返回 1

时间复杂度 $O(f(p) + g(n) \log n)$, $f(p)$ 为预处理组合数的复杂度, $g(n)$ 为单次求组合数的复杂度

```

1 auto [inv, fact, invfa] = [](int n) {
2     vector<int> inv(n, 1), fact(n, 1), invfa(n, 1);
3     for (int i = 2; i < n; i++) {
4         fact[i] = (ll)fact[i-1] * i % mod, invfa[i] = (ll)invfa[i-1] * (inv[i] = (mod - mod / i) *
5             inv[mod % i] % mod);
6     }
7     return tuple{inv, fact, invfa};
8 } ();
9 ll C(int n, int m) { return min(n - m, m) < 0 ? 0 : (ll)fact[n] * invfa[m] % mod * invfa[n - m] % mod; }
10 ll P(int n, int m) { return min(n - m, m) < 0 ? 0 : (ll)fact[n] * invfa[n - m] % mod; }
11 ll LC(ll n, ll m) { return !m ? 1 : C(n % mod, m % mod) * LC(n / mod, m / mod) % mod; }

```

鸽巢原理

鸽巢原理: 将 $n+1$ 个物体划分为 n 组 $\Rightarrow$ 至少有一组中至少有两个物体

推广: 将 n 个物体划分为 k 组 $\Rightarrow$ 至少有一组中至少有 $\lceil \frac{n}{k} \rceil$ 个物体

- **弱化为覆盖**: 给定集合 S 的非空子集构成的簇 $\{A_1, A_2, \dots, A_k\}$, 若 $\bigcup_{i=1}^k A_i = S$ 则为覆盖, 若覆盖满足

$\forall i, j \in [1, k], i < j, A_i \cap A_j = \emptyset$ 则为划分

对于 S 的一个覆盖 $\{A_1, A_2, \dots, A_k\}$ 有至少一个集合 A_i 中至少有 $|A_i| \geq \lceil \frac{|S|}{k} \rceil$ 个元素

容斥原理

定义: U 中元素有 n 种不同属性 P_i , 满足 P_i 的元素构成集合 S_i , 集合 $N = \{1, 2, \dots, n\}$

- 由德摩根定律和定义: $|U| = |\bigcup_{i \in N} S_i| = |\bigcap_{i \in \emptyset} S_i| = |\bigcap_{i \in \emptyset} \overline{S_i}|, |\emptyset| = |\bigcup_{i \in \emptyset} S_i| = |\bigcup_{i \in \emptyset} \overline{S_i}| = |\bigcap_{i \in N} \overline{S_i}|$
- **单次使用**: 子集形式为 $O(2^n)$, 若能 $O(1)$ 递推或计算每个维度的贡献, 则维度形式为 $O(n)$, 从含义入手 $O(1)$
- **对所有状态求解**: 子集形式用高维前缀和 (SOSDP) 为 $O(n2^n)$, 维度形式 $O(n^2)$, 含义入手自行分析
- **核心公式**: $|\bigcap_{i \in N} \overline{S_i}| = |\bigcup_{i \in \emptyset} S_i| = \sum_{A \subset N} (-1)^{|A|} |\bigcap_{i \in A} S_i| = \sum_{|A|=0}^n (-1)^{|A|} \sum_{A_i < A_{i+1}}^{|\overline{A}|} |\bigcap_{i=1}^{|A|} S_{A_i}|$ ($A_i < A_{i+1}$ 表示对排序后不同的 A 做求和)
 - **证明**: 对元素 x , 设其在 $T_1, T_2, \dots, T_m$ 集合中, 其出现次数由二项式定理 $(1-1)^m$ 知为 0, 即每个元素都不出现

1. **求并集**: $|\bigcup_{i \in N} S_i| = |\bigcap_{i \in \emptyset} S_i| = \sum_{A \subset N, A \neq \emptyset} (-1)^{|A|+1} |\bigcap_{i \in A} S_i| = \sum_{|A|=1}^n (-1)^{|A|+1} \sum_{A_i < A_{i+1}} |\bigcap_{i=1}^{|A|} S_{A_i}|$
2. **求交集 (补集)**: $|\bigcap_{i \in N} S_i| = |U| - |\bigcup_{i \in N} \overline{S_i}| = \sum_{A \subset N} (-1)^{|A|} |\bigcap_{i \in A} \overline{S_i}| = \sum_{|A|=0}^n (-1)^{|A|} \sum_{A_i < A_{i+1}} |\bigcap_{i=1}^{|A|} \overline{S_{A_i}}|$

1. **含义容斥**: 直接从含义上容斥, 约数容斥为其中一种

2. **广义容斥 (二项式反演)**: 设 $\alpha(k)$ 为至少满足 k 个属性的方案之和, 其会将满足 $p \geq k$ 个属性的方案之和计算 $\binom{p}{k}$ 次, $\beta(k)$ 为至多满足 k 个属性的方案之和, 其会将满足 $p \leq k$ 个属性的方案之和计算 $\binom{k}{p}$ 次, 设 $\gamma(k)$ 为恰好满足 k 个属性的方案之和

- $\alpha(k) = \sum_{i=k}^n \binom{i}{k} \gamma(i) \Rightarrow \gamma(k) = \sum_{i=k}^n (-1)^{i-k} \binom{i}{k} \alpha(i)$ (广义容斥, 当 $k=0$ 时为核心公式)
- $\beta(k) = \sum_{i=0}^k \binom{k}{i} \gamma(i) \Rightarrow \gamma(k) = \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} \beta(i)$ (二项式反演, 也是上式的补集形式)

3. **容斥反演 (一般化)**: $f(S) = \sum_{T \subset S} g(T) \Rightarrow g(S) = \sum_{T \subset S} (-1)^{|S|-|T|} f(T)$

- **补集形式**: $f(S) = \sum_{S \subset T} g(T) \Rightarrow g(S) = \sum_{S \subset T} (-1)^{|T|-|S|} f(T)$

4. **min-max 容斥**: 对满足全序关系并且元素满足可加性的序列 $\{x_i\}$, 设其长度 n , $S = \{1, 2, \dots, n\} \Rightarrow$

- $\min_{i \in S} x_i = \sum_{T \neq \emptyset \subset S} (-1)^{|T|+1} \max_{j \in T} x_j$, $\max_{i \in S} x_i = \sum_{T \neq \emptyset \subset S} (-1)^{|T|+1} \min_{j \in T} x_j$
- $\text{lcm}_{i \in S} x_i = \prod_{T \neq \emptyset \subset S} \left(\gcd_{j \in T} x_j \right)^{(-1)^{|T|+1}}$, $\text{lcm}, \gcd, a^1, a^{-1} \Leftrightarrow \max, \min, +, - \Leftrightarrow$ 对指数做 min-max 容斥

应用:

1. **不定方程非负整数解计数**:

2. **硬币购物**: $k \in [1, 4]$ 种面值的硬币, 第 i 种面值为 C_i , $n \in [1, 10^3]$ 次询问, 给出每种硬币数量 D_i 和价格 $S \in [1, 10^5]$ 问付款方案数

- 求 $\sum_{i=1}^k C_i x_i = S, x_i \leq D_i$ 的非负整数解方案数, 对于每个 $A \subset \{1, 2, \dots, k\}$, 计算 $\sum_{i=1}^k C_i x_i = S - \sum_{a \in A} C_a (D_a + 1)$ 的非负整数解方案数。用无限背包问题预处理, 询问时用容斥, 时间复杂度为 $O(kS + 2^k n)$

3. **约数容斥**: $f(k) = \sum_{x=1}^n \sum_{y=1}^n [(x, y) = k]$, 求 $f(1)$ 到 $f(n)$, 时间复杂度 $O(n \log n)$

- 找到所有以 k 为公约数的对数, 再剔除以 k 的倍数为最大公约数的对数, 即 $f(k) = \left\lfloor \frac{n}{k} \right\rfloor^2 - \sum_{i=2}^{ik \leq n} f(ik)$

4. **完全子图染色问题 (逆应用 + 含义)**: A 与 B 对图 (不一定连通) 染色 (相邻结点须同色), 对 n 阶完全图 $G = (V, E)$, 估价函数 $F(S), S \subset E, S \neq \emptyset$ 为用 m 种颜色对 $G' = (V, S)$ 染色的总方案数, 若 $|S|$ 奇数, $A+ = F(S)$, 否则, $B+ = F(S)$, 问 $A-B$

- $\text{Ans} = \sum_{S \subset E, S \neq \emptyset} (-1)^{|S|+1} F(S)$, 记完全图第 i 条边连接 u_i, v_i , Q_i 为满足 u_i, v_i 染同种颜色的染色方案, 集合大小为方案数, $F(S) = |\bigcap_{i \in S} Q_i|$, 则 $\text{Ans} = |\bigcap_{i=1}^{|E|} Q_i|$, 等式右侧含义为至少存在俩个点同色, 全集为 $|U| = m^n$, 转化为求补集, 即俩俩异色的方案数为 A_m^n , 故 $\text{Ans} = m^n - A_m^n$

5. **DAG 图计数 (DP 优化 + 广义容斥)**: 对 $n \in [1, 5 \times 10^3]$ 个点的有向无环图计数, 对 $10^9 + 7$ 取模

1. **直接 DP**: 记 $f(i, j)$ 为 i 点的 DAG, 有 j 点入度为 0 的图的个数, 设去掉这 j 个点后, 有 k 个点度数为 0, 则 j 个点与 k 个点的连边方案有 $(2^j - 1)^k$ 种, 与其余点连边方案有 $(2^{i-j-k})^j$ 种, $\Rightarrow$

$$\blacksquare f(i, j) = \binom{i}{j} \sum_{k=1}^{i-j} (2^j - 1)^k 2^{(i-j-k)j} f(i-j, k), \text{ 时间复杂度 } O(n^3)$$

2. **放宽限制 + 容斥**: 将恰好 j 个点入度为 0 改为至少 j 个点入度为 0, 记 f_i 为 i 个点的 DAG 方案数, 则 i 个点至少 j 个点入度为 0 的方案数为 $F(i, j) = \binom{i}{j} 2^{(i-j)j} f(i-j)$, 由广义容斥 $\Rightarrow$

$$\blacksquare i \text{ 个点恰好满足 } 0 \text{ 个度数为 } 0 \text{ 的方案为 } 0 = \sum_{j=0}^i (-1)^j \binom{i}{j} F(i, j) = \sum_{j=0}^i (-1)^j F(i, j) \Leftrightarrow$$

$$\blacksquare f(i) = F(i, 0) = \sum_{j=1}^i (-1)^{j+1} F(i, j) = \sum_{j=1}^i (-1)^{j+1} \binom{i}{j} 2^{(i-j)j} f(i-j), \text{ 时间复杂度 } O(n^2)$$

6. **随机游走**: 给一颗 $n \in [1, 18]$ 个点的树, 从 x 点出发, 每次等概率随机选择一条相邻的边走过去, 有 $Q \in [1, 5000]$ 次询问, 每次询问给出集合 S , 如果从 x 出发一直随机游走, 直到点集 S 中点至少经过一次, 问期望步数, 对 998244353 取模

◦ 设 x_i 为第一次走到 i 的时间, 用 min-max 容斥得所求为

$$E(\max_{i \in S} x_i) = E\left(\sum_{T \neq \emptyset \subset S} (-1)^{|T|+1} \min_{i \in T} x_i\right) = \sum_{T \neq \emptyset \subset S} (-1)^{|T|+1} E(\min_{i \in T} x_i), \text{ 考虑对于集合 } T \in [n] \text{ 求出 } F(T) = E(\min_{i \in T} x_i)$$

- 设 $f(i)$ 表示从结点 i 出发, 第一次走到 T 中某个结点的最小期望时间: $f(u) = \begin{cases} 0, & u \in T \\ 1 + \frac{1}{\deg(u)} \sum_{(u,v) \in E} f(v), & u \notin T \end{cases}$

1. **高斯消元**: 若直接求 $f(i)$, 复杂度 $O(n^3)$, 对每个 T 求 $F(T)$ 总复杂度 $O(2^n n^3)$ 不能接受

2. **树上消元**: 设根结点为 x , 结点 u 父亲 fa_u , 因为对于叶子结点 u 只与 fa_u 有关, 设 $f(u) = A_u + B_u f(fa_u)$

- 对叶结点 u 考虑其儿子序列 $v_1, \dots, v_k$, 由 $f(v) = A_v + B_v f(u) \Rightarrow$

- $f(u) = 1 + \frac{1}{\deg(u)}(f(fa_u) + \sum_{i=1}^k (A_{v_i} + B_{v_i}f(u))) \Leftrightarrow$
- $f(u) = \frac{\deg(u) + \sum_{i=1}^k A_{v_i}}{\deg(u) - \sum_{i=1}^k B_{v_i}} + \frac{f(fa_u)}{\deg(u) - \sum_{i=1}^k B_{v_i}}$ (对于叶结点也适用), 由于根 x 无父亲, 故 $F(T) = A_x$, 总复杂度 $O(2^n n)$
- 设 $F'(T) = (-1)^{|T|+1}F(T)$, 则 $E(\max_{i \in S} x_i) = \sum_{T \neq \emptyset \subset S} F'(T)$ 使用高维前缀和 (SOSDP) 在 $O(2^n n)$ 预处理对每个 S 的 $E(\max_{i \in S} x_i)$

康托展开

作用: 用来求一个 $1 \sim n$ 任意排列的排名

实现: 越靠前的数字优先级越高。也就是说如果两个排列的某一位之前的数字都相同, 那么如果这一位如果不相同, 就按这一位排序, 用数组数组或 `pbds` 统计当前有 `cnt` 个小于它的数还未出现, 则排序 $res += cnt \times (n - i)!$, 最后要加上自己, 时间复杂度 $O(n \log n)$

```
1 int Cantor = [&](int n, vector<int> & p, int res = 1) { // 最后要加上自己
2     rb_tree<int> tr;
3     for (int i = 1; i <= n; i++) {
4         tr.insert(p[i]); // (< p[i] 未出现个数) = (p[i] - 1) - (< p[i] 已经出现个数)
5         (res += (p[i] - 1 - tr.order_of_key(p[i])) * fact[n - i]) %= mod;
6     }
7     return res;
8 } ();
```

逆康托展开: 求排名为 k 的排列, k 可能很大, 令 $k = \sum_{i=1}^k S_i \times (k - i)!, S_i \in [0, k - i]$, 每次查询 rank 为 $S_i + 1$ 的数即可, 时间复杂度 $O(n \log n)$

```
1 auto inv_Cantor = [&](int n, vector<int> & s, vector<int> p = {0}) {
2     rb_tree<int> tr;
3     for (int i = 1; i <= n; i++) {
4         tr.insert(i);
5     }
6     for (int i = 1; i <= n; i++) {
7         auto it = tr.find_by_order(s[i]);
8         p.push_back(*it), tr.erase(it);
9     }
10    return p;
11 } (n, s);
```

斐波那契数列

斐波那契数列: $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}, \{0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, \dots\}$

斐波那契数列通项公式: $F_n = \frac{(\frac{1+\sqrt{5}}{2})^n - (\frac{1-\sqrt{5}}{2})^n}{\sqrt{5}} = \left\lfloor \frac{(\frac{1+\sqrt{5}}{2})^n}{\sqrt{5}} \right\rfloor$ (中括号表示离他最近的整数)

卢卡斯数列: $L_0 = 2, L_1 = 1, L_n = L_{n-1} + L_{n-2}, \{2, 1, 3, 4, 7, 11, 18, 29, 47, 76, 123, 199, \dots\}$

卢卡斯数列通项公式: $L_n = (\frac{1+\sqrt{5}}{2})^n + (\frac{1-\sqrt{5}}{2})^n$

- **关联:** $\frac{L_n + F_n \sqrt{5}}{2} = (\frac{1+\sqrt{5}}{2})^n, L_n^2 - 5F_n^2 = -4$

- **性质:**

1. **卡西尼性质:** $F_{n-1}F_{n+1} - F_n^2 = (-1)^n$
2. **附加性质:** $F_{n+m} = F_{n-1}F_m + F_nF_{m+1}$, 特别地, 当 $m = n, F_{2n} = F_n(F_{n-1} + F_{n+1}) \Rightarrow \forall k \in \mathbb{N}, F_n \mid F_{nk}$
3. **上述性质可逆:** $\forall F_a \mid F_b, a \mid b$
4. **可倍增性:** $F_{2k+1} = F_{k+1}^2 + F_k^2$
5. **前缀和:** $\sum_{i=1}^n F_i = F_{n+2} - F_1$
6. **最大公约数性质:** $\forall a, b \in \mathbb{N}, \gcd(F_a, F_b) = F_{\gcd(a, b)}$
7. **以斐波那契数列相邻两项作为输入会使欧几里得算法达到最坏复杂度:**

- **斐波那契编码:** 任何正整数都可以表示成若干个不连续的斐波那契数之和 (贪心)

- **模意义下周期性:** 抽屉原理证明每个数对 $[F_i \ F_{i+1}]$ 在第 $p^2 + 1$ 数对中必定有俩个相同的数对, 即有周期性

- **皮萨诺周期:** 模 m 意义下的最小正周期, 不超过 $6m$, 当前仅当 $m = 2 \times 5^k$ 时为 $6m$, 否则不超过 $4m$

1. $(a, b) = 1 \Rightarrow ab$ 的皮萨诺周期为 a 和 b 的皮萨诺周期的最小公倍数

- 素数 $p \equiv \pm 1 \pmod{5}$, $p-1$ 是斐波那契数模 p 的周期, 即奇素数 p 的皮萨诺周期整除 $p-1$
- 素数 $p \equiv \pm 2 \pmod{5}$, $2p+2$ 是卢卡斯数模 p 的周期, 即奇素数 p 的皮萨诺周期整除 $2p+2$
- $\forall p \in \mathbb{P}, m$ 为 $F_n \pmod{p}$ 的循环届, M 是 $F_n \pmod{p^k}$ 的循环节 $\Rightarrow M \mid mp^{k-1}$
- 实现:** 时间复杂度均为 $O(\log n)$
 - 矩阵形式:** $\begin{bmatrix} F_{n-1} & F_n \end{bmatrix} = \begin{bmatrix} F_{n-2} & F_{n-1} \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$, 设 $P = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} F_n & F_{n+1} \end{bmatrix} = \begin{bmatrix} F_0 & F_1 \end{bmatrix} P^n$
 - 快速倍增法:** 由 $F_{2k} = F_k(2F_{k+1} - F_k)$, $F_{2k+1} = F_{k+1}^2 + F_k^2$ (常数小比矩乘小)

```

1  auto fib = [] (auto && self, int n, array<int, 2> f = {0, 1}) -> array<int, 2> {
2      if (!n) return f; // return {fib[n], fib[n+1]};
3      f = self(self, n >> 1); // 注意乘法是否溢出
4      f = {(2 * f[1] + mod - f[0]) % mod * f[0] % mod, (f[1] * f[1] + f[0] * f[0]) % mod};
5      if (n & 1) return {f[1], (f[0] + f[1]) % mod};
6      return f;
7  };
8  auto Pisano_True = [] (int mod, vector<array<int, 2>> & pri_fac, int res = 1) {
9      for (auto & [p, cnt] : pri_fac) {
10         res = lcm(res, binpow(p, cnt - 1, mod) * [&](int x) -> int {
11             for (auto [t, cnt] : Prime_Factor(x)) { // 枚举因数并检查是否合法
12                 while (cnt-- && fib(fib, x / t, p) == fib(fib, 0, p)) x /= t;
13             }
14             return x;
15         } (vector{p << 2, p - 1, p + 1 << 1, p + 1 << 1, p - 1}[p % 5]));
16     }
17     return res;
18 };
19 auto Pisano_False = [] (int mod, vector<array<int, 2>> & pri_fac, int res = 1) {
20     for (auto & [p, cnt] : pri_fac) { // 当 p 为 2 可以特判周期为 3
21         res = lcm(res, binpow(p, cnt - 1, mod) * vector{p << 2, p - 1, p + 1 << 1, p + 1 << 1, p - 1}[p %
22         5]);
23     }
24     return res;
25 };

```

错位排列

没有任何元素出现在其有序位置的排列, 记其方案数为 D_n , $D_0 = 1, D_1 = 0, D_2 = 1, \{0, 1, 2, 9, 44, 265, \dots\}$

容斥原理计算:

全集为 $n!$ 属性为 $P_i \neq i$ 转化为 $|\bigcup_{i=1}^n S_i|$, 记 s_i 为满足 $P_i = i$ 的排列数量, $|\bigcap_{i=1}^m S_{a_i}| = (n-m)!$

$$\begin{aligned}
 |\bigcup_{i=1}^n S_i| &= \sum_{m=1}^n (-1)^{m-1} \sum_{a_i < a_{i+1}} |\bigcap_{i=1}^m S_{a_i}| \\
 &= \sum_{m=1}^n (-1)^{m-1} \binom{n}{m} (n-m)! \\
 &= n! \sum_{m=1}^n \frac{(-1)^{m-1}}{m!}
 \end{aligned}$$

$$\text{故 } D_n = n! - |\bigcup_{i=1}^n S_i| = n! - n! \sum_{m=1}^n \frac{(-1)^{m-1}}{m!} = n! \sum_{m=0}^n \frac{(-1)^m}{m!}$$

递推关系:

动态规划思想推出 $D_n = (n-1)(D_{n-1} + D_{n-2})$

进一步推得 $D_n = nD_{n-1} + (-1)^n$

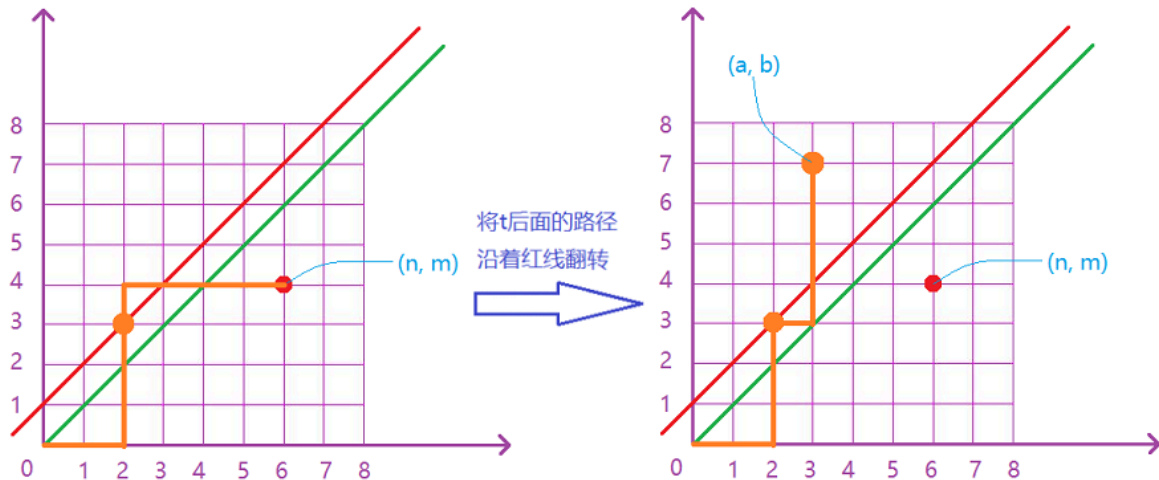
其他关系:

$D_n = \lfloor \frac{n!}{e} \rfloor$ 增长速度与阶乘仅相差常数

$$\text{形成错位的概率 } P = \lim_{n \rightarrow \infty} \frac{D_n}{n!} = \frac{1}{e}$$

卡特兰数

卡特兰数的推导：



$\forall n, m \in \mathbb{Z}, n \geq m$, 从 $(0, 0)$ 走到 (n, m) 且不超过直线 $y = x$ 的方案数为 $2\binom{m+n}{n} - 2\binom{m+n}{m-1}$

$\Leftrightarrow$ 从 $(0, 0)$ 走到 (n, m) 不经过直线 $y = x + 1$ 的方案数 $\Leftrightarrow$ 从 $(0, 0)$ 走到 (n, m) 关于 $y = x + 1$ 对称的点 $(m - 1, n + 1)$ 的方案数, 故直线下方路径方案数为 $\binom{m+n}{n} - \binom{m+n}{m-1}$, 再由对称性为 $2\binom{m+n}{n} - 2\binom{m+n}{m-1}$

• **定义：** 当 $n = m$ 时方案数为卡特兰数, 用于解决有顺序且一对匹配的组数问题计数问题

• **括号问题：** $2n$ 个括号, n 对正确括号序列个数, $\{1, 1, 2, 5, 14, 42, 132, \dots\}$

- $H_n = \frac{\binom{2n}{n}}{n+1}, n \in \mathbb{N}$
- $H_n = \begin{cases} \sum_{i=0}^{n-1} H_i H_{n-i-1} & n \geq 1, n \in \mathbb{N}_+ \\ 1 & n = 0 \end{cases}$
- $H_n = \frac{H_{n-1}(4n-2)}{n+1}$
- $H_n = \binom{2n}{n} - \binom{2n}{n-1}$
- $H(x) = \sum_{n \geq 0} \binom{2n}{n} \frac{1}{n+1} x^n$

• **应用：**

1. $2n$ 个人排队, 入场费 5 元, n 人有一张 5 元, n 人有一张 10 元, 剧院无其他钞票, 问多少种方法使得剧院时刻能找零
2. **路径计数问题：** 非降路径是指只能向上或向右走的路径
 - $n \times n$ 方格图, 从 $(0, 0)$ 走到 (n, n) , 求不穿过对角线的方案数 (括号序列) (**数值法：** $\text{Ans} = 2H_n = \binom{2n}{n} - \binom{2n}{n-1}$)
 - 从 $(0, 0)$ 走到 (n, n) , 求除端点外不接触直线 $y = x$ 方案数, 先求直线下方路径 $\Rightarrow$ 从 $(1, 0)$ 到 $(n, n-1)$ 的方案数为 H_{n-1} , 对称性有 $\text{Ans} = 2H_{n-1}$
3. n 个点的二叉树的形态数 $\Leftrightarrow$ 求 $2n + 1$ 个点除叶结点都有左右儿子的二叉树, 最后去除 $n + 1$ 个叶结点, dfn 为 $2n$ 的括号序列
 - **数值法：** $H(n)$ 表示 n 个节点的二叉树的形态数, $H(n) = \sum_{i=0}^{n-1} H_i H_{n-i-1}$
4. 圆上选 $2n$ 个点, 这些点对连接起来得到 n 条线段不相交的方案数, 直接为括号序列
5. 对角线不相交情况下, 将一个凸包分成三角形区域的方案数, 同上
6. 一个进栈序列 $1, 2, \dots, n$, 求出栈序列的方案数, 直接为括号序列
7. 由 n 个 $+1$ 和 n 个 -1 组成的序列, 使得任意前缀和非负的方案数, 直接为括号序列

斯特林数

第二类斯特林数 (斯特林子集数)： $\{n\}_k \Leftrightarrow S(n, k)$, 表示将 n 个两两不同的元素, 划分为 k 个互不区分的非空子集的方案数

1. **递推式：** $\{n\}_k = \begin{cases} \{n-1\}_k + k\{n-1\}_{k-1} & , otherwise \\ [n=0] & , k=0 \end{cases}$
2. **通项公式：** $\{n\}_k = \sum_{i=0}^k \frac{(-1)^{k-i} i^n}{i!(k-i)!}$

第一类斯特林数 (斯特林轮换数)： $[n]_k \Leftrightarrow s(n, k)$, 表示将 n 个两两不同的元素, 划分为 k 个互不区分的非空轮换的方案数

1. **递推式：** $[n]_k = \begin{cases} [n-1]_k + (n-1)[n-1]_{k-1} & , otherwise \\ [n=0] & , k=0 \end{cases}$

线性代数

线性基

定义：线性空间 V 的一个极大线性无关组为 V 的一组线性基，规定 $\{\emptyset\}$ 为空集， V 的维数为 $\dim V$ 为基的元素个数

性质：

(i) $\forall$ 有限维线性空间 V , $\dim V = n$

- (a) V 中任意 $n + 1$ 个向量线性相关
- (b) V 中任意 n 个线性无关的向量均为 V 的基
- (c) V 中任意向量可被 $a_1, a_2, \dots, a_b$ 线性表出 $\Leftrightarrow$ 其是 V 的一个基

(ii) V_1, V_2 是有限维线性空间, $V_1 + V_2$ 和 $V_1 \cap V_2$ 也为有限维

- (a) $\dim V_1 + \dim V_2 = \dim(V_1 + V_2) + \dim(V_1 \cap V_2)$
- (b)

$V_1 + V_2 = V_1 \oplus V_2 \Leftrightarrow \dim V_1 + \dim V_2 = \dim(V_1 + V_2) \Leftrightarrow a_1, \dots, a_n$ 为 V_1 的基, $b_1, \dots, b_m$ 为 V_2 的基, 则 $a_1, \dots, a_n, b_1, \dots, b_m$ 为 $V_1 + V_2$ 的基

(iii) 求异或为 x 的方案数为 $2^{\text{自由变量个数}}$

贪心法求线性基： n 为向量长度, m 为向量总数, 时间复杂度 $O(nm)$

```
1  struct LiBase : array<int, 64> {
2      int rank;
3      LiBase(int rk = 0) : array(), rank(rk) {}
4
5      int Insert(int x) {
6          for (int i = size() - 1; ~i; i--) {
7              if (x >> i) if (!at(i)) return rank++, at(i) = x;
8              else x ^= at(i);
9          }
10         return 0;
11     }
12     void gauss() { //将基化为阶梯矩阵
13         for (int i = 0; i < size(); i++) {
14             for (int j = i + 1; j < size(); j++) {
15                 if ((at(j) >> i & 1) && at(i)) at(j) ^= at(i);
16             }
17         }
18     }
19     int find_max(int res = 0) { //求异或最大值
20         for (auto & x : *this) {
21             res = max(res, res ^ x);
22         }
23         return res;
24     }
25 };
```

线性基加LCA

维护每个节点到根节点的线性基，但是贪心的使得深度更大的点成为线性基

每次查询的时候只合并深度大于等于 lca 深度的线性基

```
1  struct line_base {
2      array<int, 64> p = {0};
3      vector<int> path;
4      int rank = 0;
5
6      int insert(int x, int v) {
7          for (int i = p.size() - 1; ~i; i--) {
8              if (x >> i) {
9                  if (!p[i]) {
10                     p[i] = x;
11                     rank++;
12                     path.push_back(v);
13                     return 1;
14                 }
15                 x ^= p[i];
16             }
17         }
18         return 0;
19     }
```

```

19     }
20 };
21
22 auto dfs = [&](auto dfs, int u, int father) -> void {
23     p[u].insert(a[u], u); //更改插入顺序使得深度更深的成为线性基
24     for (auto x : p[father].path) {
25         p[u].insert(a[x], x);
26     }
27
28     for (int i = 1; i < 31; i++) { //倍增算出父亲
29         fa[u][i] = fa[fa[u][i - 1]][i - 1];
30     }
31
32     for (auto v : adj[u]) { //dfs
33         if (v == father) continue;
34         fa[v][0] = u, dep[v] = dep[u] + 1; //记录
35         dfs(dfs, v, u);
36     }
37 };
38
39 for (int i = 1; i <= q; i++) {
40     int x, y;
41     cin >> x >> y;
42
43     int tmp = lca(x, y);
44
45     line_base t = {};
46
47     for (auto i : p[x].path) {
48         if (dep[i] >= dep[tmp]) {
49             t.insert(a[i], i);
50         }
51     }
52     for (auto i : p[y].path) {
53         if (dep[i] >= dep[tmp]) {
54             t.insert(a[i], i);
55         }
56     }
57
58     cout << t.find_max() << "\n";
59 }

```

矩阵

$n \times n$ 01 矩阵, 每个 01 都是连续区间, 高斯消元为 $O(n \log n)$, 每次消完都是一个连续区间, 记录右端点即可

```

1  template <int mod = mod>
2  struct matrix : vector<vector<int>> { // 下标从 0 开始
3      int n, m, rank; // 行, 列, 秩
4      matrix(int r, int c, int x = 0, int rk = 0) : vector(r, vector<int>(c)), n(r), m(c), rank(rk) {
5          if (x) for (assert(n == m); r; at(--r)[r] = x); // 对角矩阵, x = 1 为幺元, x = 0 为零元, !! 需要 n = m
6      }
7      matrix(initializer_list<initializer_list<int>> t) : matrix(t.size(), t.size() ? t.begin() -> size() : 0) {
8          for (auto &x : t) {
9              for (auto &y : x) {
10                 at(&x - t.begin())[&y - x.begin()] = y;
11             }
12         }
13     }
14
15     matrix operator * (const matrix &t) { // 矩阵乘法, 有取mod
16         assert(m == t.n); matrix res(n, t.m);
17         for (int i = 0; i < n; i++) {
18             for (int k = 0; k < m; k++) { // 此枚举顺序为常数优化
19                 for (int j = 0, tmp = at(i)[k]; j < t.m; (res[i][j++] += t[k][j] * tmp) %= mod);
20             }
21         }
22         return res;
23     } matrix & operator *= (const matrix &t) { return *this = *this * t; }
24     matrix & operator *= (const int &k) {
25         for (int i = 0; i < n; i++) {
26             for (int j = 0; j < m; (at(i)[j++] *= k) %= mod);
27         }
28     }
29 }

```

```

28     return *this;
29 } matrix operator * (const int & k) { return matrix(*this) *= k; }
30
31 matrix & operator += (const matrix & t) { // 矩阵加法,有取mod
32     assert(n == t.n && m == t.m);
33     for (int i = 0; i < n; i++) {
34         for (int j = 0; j < m; (at(i)[j++] += t[i][j]) %= mod);
35     }
36     return *this;
37 } matrix operator + (const matrix & t) { return matrix(*this) += t; }
38 matrix & operator -= (const matrix & t) {
39     assert(n == t.n && m == t.m);
40     for (int i = 0; i < n; i++) {
41         for (int j = 0; j < m; (at(i)[j++] = mod - t[i][j]) %= mod);
42     }
43     return *this;
44 } matrix operator - (const matrix & t) { return matrix(*this) -= t; }
45
46 matrix pow(int b) {
47     assert(n == m); matrix a(*this), res(a.n, a.m, 1);
48     for (; b; b >>= 1, a *= a) {
49         if (b & 1) res *= a;
50     }
51     return res;
52 }
53 matrix Reverse() {
54     matrix a(m, n);
55     for (int i = 0; i < n; i++) {
56         for (int j = 0; j < m; a[j++][i] = at(i)[j]);
57     }
58     return a;
59 }
60 matrix inv() {
61     return assert(n == m), gauss(matrix(n, n, 1)); // 只有可能无解,不可能有无穷解
62 }
63
64 matrix gauss(const matrix & t) { // b[0][0] = -1 无解, b[0][0] = 1 无穷解, b[0][0] = 0 唯一解
65     matrix b(t), a(*this);
66     for (int r = 0, c = 0, t; r < n && c < m; c++) {
67         if ((t = find_if(a.begin() + r, a.end(), [&](auto & x) { return x[c]; }) - a.begin()) >= n)
68             continue;
69         b[rank = r++].swap(b[t]), a[rank].swap(a[t]), t = binpow(a[rank][c], mod - 2, mod);
70         for (int i = b.m; i; (b[rank][--i] *= t) %= mod);
71         for (int i = m; i > c; (a[rank][--i] *= t) %= mod);
72         for (int i = r; i < n; i++) {
73             for (int j = b.m; j; (b[i][j] += mod - a[i][c] * b[rank][--j] % mod) %= mod);
74             for (int j = m; j > c; (a[i][j] += mod - a[i][c] * a[rank][--j] % mod) %= mod);
75         }
76         if (find_if(b.begin() + rank + 1, b.end(), [&](auto & x) { // R(A) < R(A|B) 无解
77             return find_if(x.begin(), x.end(), [&](auto & x) { return x; }) != x.end(); }) != b.end()) return
78     {};;
79     for (int i = rank++; ~i; i--) {
80         for (int j = i + 1; j < m; a[i][j++] = 0) {
81             for (int o = 0; o < b.m; (b[i][o++] += mod - a[i][j] * b[j][o] % mod) %= mod);
82         }
83     }
84     return b; // 有唯一解R(A) = R(A|B) = m, R(A) = R(A|B) < m 有无穷多解,使用前记得先判断无穷解
85 }
86
87 vector<vector<double>> double_gauss(const matrix & t) { // k 为左侧矩阵的范围,当右侧为一列时, k = m - 1
88     vector<vector<double>> b(n, vector<double>(t.m)), a(n, vector<double>(m));
89     for (int i = 0; i < n; copy(t[i].begin(), t[i].end(), b[i].begin()), copy(at(i++).begin(), at(i).end(),
90     a[i].begin()));
91     for (int r = 0, c = 0, it; r < n && c < m; c++) { // 转化为上三角矩阵,然后找主元
92         if (abs(a[it = max_element(a.begin() + r, a.end(), [&](auto & x, auto & y) { return abs(x[c]) <
93     abs(y[c]); }) - a.begin())[c]) < 1e-12) continue; // 无主元,千万记得加绝对值
94         b[rank = r++].swap(b[it]), a[rank].swap(a[it]); // 行自增,秩为 rank = r - 1,交换上去
95         for (int i = t.m; i; b[rank][--i] /= a[rank][c]);
96         for (int i = m; i > c; a[rank][--i] /= a[rank][c]); // 归一化
97         for (int i = r; i < n; i++) { // 将下面的列消为0
98             for (int j = t.m; j; b[i][j] -= a[i][c] * b[rank][--j]);
99             for (int j = m; j > c; a[i][j] -= a[i][c] * a[rank][--j]);
100         }
101     }
102 }

```

```

99         if (find_if(b.begin() + rank + 1, b.end(), [&](auto & x) { // R(A) < R(A|B) 无解, 记得绝对值
100             return find_if(x.begin(), x.end(), [&](auto & x) { return abs(x) > 1e-12; }) != x.end(); }) !=
        b.end()) return {};
101         for (int i = rank++; ~i; i--) { // 消成对角矩阵
102             for (int j = i + 1; j < m; a[i][j++] = 0) {
103                 for (int o = 0; o < t.m; (b[i][o++] -= a[i][j] * b[j][o]));
104             }
105         }
106         return b; // 有唯一解R(A|B) = 变元数 = m, R(A|B) < m 有无穷多解
107     }
108
109     int delta(int res = 1, int sign = 0) { // 适用于任意模数和没逆元的情况
110         assert(n == m); matrix a(*this);
111         for (int r = 0, t; r < n; (res *= a[r][r++]) %= mod) {
112             if (a[t = max_element(a.begin() + r, a.end(), [&](auto & x, auto & y) { return x[r] < y[r]; }) -
        a.begin())[r] == 0) return 0;
113             if (r != t) a[r].swap(a[t]), sign ^= 1;
114             for (int i = r + 1; i < n; i++) {
115                 if (a[i][r] > a[r][r]) a[r].swap(a[i]), sign ^= 1;
116                 for ( ; a[i][r]; a[r].swap(a[i]), sign ^= 1) {
117                     for (int j = r, t = a[r][r] / a[i][r]; j < m; (a[r][j++] += (mod - t) * a[i][j]) %= mod);
118                 }
119             }
120         }
121         return sign ? (mod - res) % mod : res;
122     }
123
124     double double_delta(int res = 1) { // 计算行列式
125         assert(n == m); vector<vector<double>>> a(n, vector<double>(m));
126         for (int i = 0; i < n; copy(at(i++).begin(), at(i).end(), a[i].begin()));
127         for (int r = 0, t; r < n; r++) {
128             if (abs(a[t = max_element(a.begin() + r, a.end(), [&](auto & x, auto & y) { return abs(x[r]) <
        abs(y[r]); }) - a.begin())[r] < 1e-12) return 0;
129             if (r != t) a[r].swap(a[t]), res *= -1;
130             res *= a[r][r];
131             for (int i = n; i > r; a[r][--i] /= a[r][r]);
132             for (int i = r + 1; i < n; i++) {
133                 for (int j = n; j > r; a[i][j] -= a[i][r] * a[r][--j]);
134             }
135         }
136         return res;
137     };

```

矩阵快速幂

矩阵快速幂：用二进制计算矩阵的幂次,即计算 A^k ,特别的, n 阶矩阵 $A^0 = I_n$

多用于加速递推

矩阵乘法：时间复杂度 $O(n^3)$

矩阵乘法加快速幂：时间复杂度 $O(m^3 \log n)$

应用：利用矩阵快速幂加速数列的计算,当 dp 转移方程不多时可用于 dp 转移加速,如：斐波那契数列

$$\text{斐波那契数列 } a_n = \begin{cases} 1 & n=1, n=2 \\ a_{n-1} + a_{n-2} & n>2 \end{cases}$$

$$\text{记 } 1 \times 2 \text{ 阶矩阵 } F(n) = [a_n \quad a_{n-1}] \Rightarrow F(n-1) = [a_{n-1} \quad a_{n-2}]$$

$$\text{试设矩阵 } B \text{ 使得 } F(n-1) \times B = F(n-1) \Rightarrow B = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$$

$$\text{故 } F(n) = F(2) \times B^{n-2} \Rightarrow F(n) \text{ 的第一行第一列元素即为 } a_n$$

B^{n-2} 可用矩阵快速幂求解,故总时间复杂度 $O(\log n)$

斐波那契结论：斐波那契的前缀和 $pre(n) = f(n+2) - 1$

高斯消元

初等行列变换：

(i) 交换某两行,解不变

(ii) 把某一行乘非零数,解不变

(iii)把某行的若干倍加到另一行去,解不变

高斯消元五步骤:

(i) 增广矩阵初等行变换为最简形

(ii) 还原线性方程组

(iii)求解第一个变量

(iv)补充自由未知量

(v) 列表示方程组通解

判断线性方程组的解:

系数矩阵的秩 $R(A)$ = 增广矩阵的秩 $R(A|B) \Leftrightarrow$ 线性方程组有解

系数矩阵的秩 $R(A)$ = 增广矩阵的秩 $R(A|B)$ = 变元数 $n \Leftrightarrow$ 线性方程组有唯一解

系数矩阵的秩 $R(A)$ = 增广矩阵的秩 $R(A|B) <$ 变元数 $n \Leftrightarrow$ 线性方程组有无穷多组解

系数矩阵的秩 $R(A) <$ 增广矩阵的秩 $R(A|B) \Leftrightarrow$ 线性方程组无解

时间复杂度: $O(m^2n)$, m 为列, n 为方程数

高斯消元解异或方程组:

由于系数和解均为0/1, 使用`bitset`优化常数

时间复杂度: $O(\frac{m^2n}{w})$

```
1  auto xor_gauss = [&](int n, int m, vector<bitset<110>> & Matrix) { //n为行,m为列
2      auto a = Matrix;
3      int r = 1;
4
5      for (int c = 1; c < m && r <= n; c++) { //转化为上三角矩阵
6          int t = r; //找主元
7          for (int i = r + 1; i <= n; i++)
8              if (a[i][c]) t = i; //主元
9          if (!a[t][c]) continue; //无主元
10         swap(a[r], a[t]); //交换 //不需要归一化
11         for (int i = r + 1; i <= n; i++) //将下面的列消为0
12             if (a[i][c])
13                 a[i] ^= a[r];
14         r++; //行自增
15     }
16     r--; //此时r为秩
17
18     for (int i = r + 1; i <= n; i++)
19         if (a[i][m])
20             return pair{(int)0, a}; //R(A) < R(A|B) 无解
21
22     if (r < m - 1) //R(A|B)小于变元数
23         return pair{(int)2, a}; //R(A) = R(A|B) 有2^(m - 1 - r)个解
24
25     for (int i = m - 1; i; i--) //m - 1为变元数
26         for (int j = i + 1; j <= m - 1; j++)
27             a[i][m] = a[i][m] ^ (a[i][j] & a[j][m]), a[i][j] = 0;
28     return pair{(int)1, a}; //有唯一解R(A|B) = 变元数
29 };
```

概率论

概率与期望

线性性质: $E(aX + bY) = aE(X) + bE(Y)$, X, Y 为事件

概率dp: 一般从前往后

期望dp: 利用线性性质, 计算期望时一般由终点状态倒推回起点

条件概率与独立性

贝叶斯公式：
$$P(A_i|B) = \frac{P(A_iB)}{P(B)} = \frac{P(A_i)P(B|A_i)}{\sum_{j=1}^n P(A_j)P(B|A_j)}$$

博弈论

公平组合游戏

Nim游戏

意是给出n堆石子，轮流在其中一堆去任意个，谁不能去谁就输。

思路：证明所有石子异或和为0则先手必输

- 1. 最后情况是每堆均为0, 先手必输
- 2. 若异或和最高位为*i*, 则一定有一堆石子第*i*位为1
- 3. 设这堆石子为*A1*, 其他石子异或和为*x*, 总异或和为*k*, 即 $A1 \oplus x = k$
则先手的人可以将异或和变为0 $\Rightarrow (A1 \oplus k) \oplus x = 0$
- 4. 而后手的人无法将异或和保持为0
- 5. 故先手的人可以一直将后手的人的异或和保持为0, 而游戏次数有限, 故一定后手必败

有向图游戏与SG函数：

定义： $\forall x$ 和他所有 k 个后继状态 $y_1, y_2, \dots, y_k, SG(x) = mex\{SG(y_1), SG(y_2), \dots, SG(y_k)\}$

$\forall n$ 个有向图游戏组成的组合游戏, 起点为 $s_1, s_2, \dots, s_n$ 先手必胜 $\Leftrightarrow SG(s_1) \oplus SG(s_2) \oplus \dots \oplus SG(s_n) \neq 0$

二分图博弈：

一定在二分图最大匹配的状态先手必胜

不公平组合游戏

反常游戏

博弈模型：

一. 巴什博弈（Bash Game）：

A和B一块报数，每人每次报最少1个，最多报4个，看谁先报到30。这应该是最古老的关于巴什博弈的游戏了吧。

其实如果知道原理，这游戏一点运气成分都没有，只和先后手有关，比如第一次报数，A报*k*个数，那么B报5-*k*个数，那么B报数之后问题就变为，A和B一块报数，看谁先报到25了，进而变为20,15,10,5，当到5的时候，不管A怎么报数，最后一个数肯定是B报的，可以看出，作为后手的B在游戏中是不会输的。

那么如果我们要报*n*个数，每次最少报一个，最多报*m*个，我们可以找到这么一个整数*k*和*r*，使 $n = k * (m + 1) + r$ ，代入上面的例子我们就可以知道，如果*r*=0，那么先手必败；否则，先手必胜。

二. 威佐夫博弈（Wythoff Game）：

有两堆各若干的物品，两人轮流从其中一堆取至少一件物品，至多不限，或从两堆中同时取相同件物品，规定最后取完者胜利。

直接说结论了，若两堆物品的初始值为（*x*, *y*），且*x*<*y*，则另*z*=*y*-*x*；

记 $w = (\text{int}) [((\text{sqrt}(5) + 1) / 2) * z]$ ；

若*w*=*x*，则先手必败，否则先手必胜。

三. 尼姆博弈（Nimm Game）：

尼姆博弈指的是这样一个博弈游戏：有任意堆物品，每堆物品的个数是任意的，双方轮流从中取物品，每一次只能从一堆物品中取部分或全部物品，最少取一件，取到最后一件物品的人获胜。

结论就是：把每堆物品数全部异或起来，如果得到的值为0，那么先手必败，否则先手必胜。

四. 斐波那契博弈：

有一堆物品，两人轮流取物品，先手最少取一个，至多无上限，但不能把物品取完，之后每次取的物品数不能超过上一次取的物品数的二倍且至少为一件，取走最后一件物品的人获胜。

结论是：先手胜当且仅当n不是斐波那契数（n为物品总数）

序理论

偏序集合：

- 自反性： $a \leq a$
- 反对称性： $a \leq b \wedge b \leq a \Rightarrow a = b$
- 传递性： $a \leq b \wedge b \leq c \Rightarrow a \leq c$

全序关系：在偏序关系上多一个限制, $\forall a, b \in P, a \leq b \vee b \leq a$

严格弱序(C++自定义比较器要求)：

- 非自反性： $f(x, x)$ 为假 $\Leftrightarrow x \not< x$
- 非对称性： $f(x, y)$ 为真 $\Rightarrow f(y, x)$ 为假 $\Leftrightarrow x < y \Rightarrow y \not< x$
- 传递性： $f(x, y)$ 为真 $\wedge f(y, z)$ 为真 $\Rightarrow f(x, z)$ 为真 $\Leftrightarrow x < y \wedge y < z \Rightarrow x < z$
- 不可比的传递性：
 $f(x, y)$ 为假 $\wedge f(y, x)$ 为假 $\wedge f(y, z)$ 为假 $\wedge f(z, y)$ 为假 $\Rightarrow f(x, z)$ 为假 $\wedge f(z, x)$ 为假 $\Leftrightarrow x \not< y \wedge y \not< x \wedge y \not< z \wedge z \not< y \Rightarrow x \not< z \wedge z \not< x$